

CLC Compiler v4.0

LogiCORE IP Product Guide

Vivado Design Suite

PG140 (v4.0) July 22, 2026



Table of Contents

Chapter 1: Introduction.....	4
Features.....	4
IP Facts.....	5
Chapter 2: Overview.....	6
Navigating Content by Design Process.....	6
Core Overview.....	6
Licensing and Ordering.....	7
Chapter 3: Product Specification.....	8
Performance.....	8
Port Descriptions.....	8
Event Signals.....	10
Chapter 4: Designing with the Core.....	12
General Design Guidelines.....	12
Frequency Response Characteristics.....	13
CIC Decimator.....	18
CIC Interpolator.....	21
Output Width and Gain.....	23
Clocking.....	23
Resets.....	23
Protocol Description.....	24
Chapter 5: Design Flow Steps.....	31
Customizing and Generating the Core.....	31
Constraining the Core.....	38
Simulation.....	38
Synthesis and Implementation.....	39
Chapter 6: C Model.....	40
Unpacking and Model Contents.....	40



- Installation..... 41
- C Model Interface.....42
- Compiling..... 46
- Linking..... 46
- Example..... 47
- MATLAB Interface.....47
- Chapter 7: Test Bench.....51**
 - Demonstration Test Bench..... 51
- Appendix A: Upgrading..... 53**
 - Migrating to the Vivado Design Suite..... 53
 - Upgrading in the Vivado Design Suite.....53
- Appendix B: Debugging.....63**
 - Finding Help with AMD Adaptive Computing Solutions.....63
 - Debug Tools..... 64
 - Interface Debug..... 65
- Appendix C: Additional Resources and Legal Notices..... 66**
 - Finding Additional Documentation.....66
 - Support Resources.....67
 - References.....67
 - Revision History.....67
 - Please Read: Important Legal Notices..... 68

Introduction

The AMD LogiCORE™ IP CIC Compiler core provides the ability to design and implement AXI4-Stream-compliant cascaded integrator-comb (CIC) filters.

Features

- AXI4-Stream-compliant interfaces
- Decimation or interpolation
- Fixed or programmable rate change from 4 to 8192
- Three to six CIC stages
- One or two differential delays
- Support of signed two's-complement input data from 2 bits to 32 bits
- Full or limited precision output data
- Single or multichannel support for up to 16 channels
- Hardware folding for small-footprint implementations
- Optional mapping to DSP48E1 slices
- Synchronous clear input
- Clock enable input
- Use with AMD Vivado™ IP catalog and AMD System Generator for DSP

IP Facts

AMD LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family ¹	AMD Versal™ adaptive SoCs, AMD UltraScale+™, AMD UltraScale™, AMD Zynq™ 7000 SoC, 7 series
Supported User Interfaces	AXI4-Stream
Resources	Performance and Resource Utilization web page
Provided with Core	
Design Files	Encrypted RTL
Example Design	Not Provided
Test Bench	VHDL
Constraints File	Not Applicable
Simulation Model	Encrypted VHDL
Supported S/W Driver	N/A
Tested Design Flows ²	
Design Entry	AMD Vivado™ Design Suite, System Generator for DSP
Simulation	For supported simulators, refer to the <i>Vivado Design Suite User Guide: Release Notes, Installation, and Licensing</i> (UG973).
Synthesis	Vivado Synthesis
Support	
Release Notes and Known Issues	Master Answer Record: 29297
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Support web page	

Notes:

1. For a complete list of supported devices, refer to the AMD Vivado™ IP catalog.
2. For the supported versions of third-party tools, refer to the *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* (UG973).

Overview

Navigating Content by Design Process

AMD Adaptive Computing documentation is organized around a set of standard design processes to help you find relevant content for your current development task. You can access the AMD Versal™ adaptive SoC design processes on the [Design Hubs](#) page. You can also use the [Design Flow Assistant](#) to better understand the design flows and find content that is specific to your intended design needs. This document covers the following design processes:

- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the AMD Vivado™ timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
 - [Port Descriptions](#)
 - [Clocking](#)
 - [Resets](#)
 - [Customizing and Generating the Core](#)
-

Core Overview

Cascaded Integrator-Comb (CIC) filters, also known as Hogenauer filters, are multirate filters often used for implementing large sample rate changes in digital systems. They are typically employed in applications that have a large excess sample rate. That is, the system sample rate is much larger than the bandwidth occupied by the processed signal as in digital down converters (DDCs) and digital up converters (DUCs). Implementations of CIC filters have structures that use only adders, subtractors, and delay elements. These structures make CIC filters appealing for their hardware-efficient implementations of multirate filtering.

Licensing and Ordering

This AMD LogiCORE™ IP module is provided at no additional cost with the AMD Vivado™ Design Suite under the terms of the [End User License](#).

Information about other AMD LogiCORE™ IP modules is available at the [Intellectual Property](#) page. For information about pricing and availability of other AMD LogiCORE IP modules and tools, contact your [local sales representative](#).

Product Specification

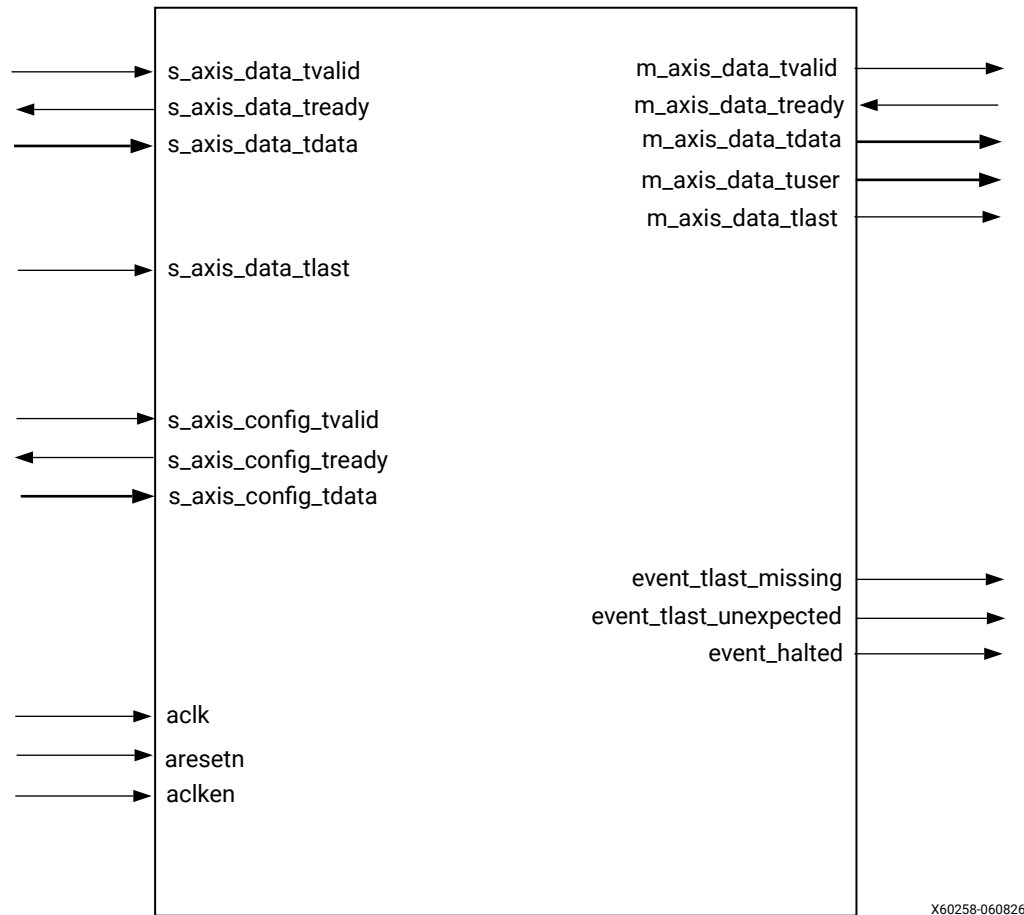
Performance

For full details about performance and resource use, visit the [Performance and Resource Use web page](#).

Port Descriptions

The core interfaces are shown in the following figure and table.

Figure 1: Core Ports



Core Signal Pinout

Name	Direction	Description
aclk	I	Rising edge clock.
aclken	I	Active-High clock enable (optional).
aresetn	I	Active-Low synchronous clear (optional, always takes priority over aclken). A minimum active pulse of two cycles is required.
s_axis_config_tvalid	I	TVALID for the Configuration Channel. Asserted by the external master to signal that it can provide data.
s_axis_config_tready	O	TREADY for the Configuration Channel. Asserted by the CIC Compiler to signal it is ready to accept configuration data.
s_axis_config_tdata	I	TDATA for the Configuration Channel. Carries configuration information (RATE).
s_axis_data_tvalid	I	TVALID for the Data Input Channel. Used by the external master to signal that data is available.
s_axis_data_tready	O	TREADY for the Data Input Channel. Used by the CIC Compiler to signal that it is ready to accept data.
s_axis_data_tdata	I	TDATA for the Data Input Channel. Carries unprocessed sample data.

Name	Direction	Description
s_axis_data_tlast	I	TLAST for the Data Input Channel. Present only in multichannel mode and asserted on the sample corresponding to the last channel. Used only to generate event_tlast_unexpected and event_tlast_missing.
m_axis_data_tvalid	O	TVALID for the Data Output Channel. Asserted by the CIC Compiler when sample data is available.
m_axis_data_tready	I	TREADY for the Data Output Channel. Asserted by the external slave when ready to accept data. Present only when HAS_DOUT_TREADY is TRUE.
m_axis_data_tdata	O	TDATA for the Data Output Channel. Carries processed sample data.
m_axis_data_tuser	O	TUSER for the Data Output Channel. Present only in multichannel mode; carries additional per-sample information (CHAN_OUT, CHAN_SYNC).
m_axis_data_tlast	O	TLAST for the Data Output Channel. Present only in multichannel mode and asserted on the sample corresponding to the last channel.
event_tlast_missing	O	Asserted when s_axis_data_tlast is not asserted on the sample corresponding to the last channel (multichannel mode only).
event_tlast_unexpected	O	Asserted when s_axis_data_tlast is asserted on a sample that does not correspond to the last channel (multichannel mode only).
event_halted	O	Asserted when the core needs to write to the Data Output Channel but cannot because buffers are full (present when HAS_DOUT_TREADY is TRUE).

Event Signals

The CIC Compiler core provides some real-time non-AXI signals to report information about the core status. These event signals are updated on a clock-cycle-by-clock-cycle basis, and are intended for use by reactive components such as interrupt controllers. These signals are not optionally configurable from the Vivado Integrated Design Environment (IDE), but are removed by synthesis tools if left unconnected.

event_tlast_missing

This event signal is asserted for a single clock cycle when s_axis_data_tlast is Low on the data sample corresponding to the last channel. This is intended to show a mismatch between the CIC Compiler and the upstream data source with regard to the channel synchronization. The event pin is only available when the core is configured to have multiple channels.

event_tlast_unexpected

This event signal is asserted for a single clock cycle when the CIC Compiler sees s_axis_data_tlast High on any incoming data sample that does not correspond to the last channel.

This signal is intended to show a mismatch between the CIC Compiler and the upstream data source with regard to channel synchronization.

If there are multiple unexpected highs on `s_axis_data_tlast`, then this signal is asserted for each of them. The event pin is only available when the core is configured to have multiple channels.

event_halted

This event is asserted on every cycle where the CIC Compiler needs to write data to the Data Output Channel but cannot because the buffers in the channel are full. When this occurs, the CIC Compiler core is halted and all activity stops until space is available in the channel buffers. The event pin is only available when `HAS_DOUT_TREADY` is TRUE.

Designing with the Core

This section includes guidelines and additional information to facilitate designing with the core.

General Design Guidelines

This description of the CIC decimator and interpolator is based closely on that provided in *An Economical Class of Digital Filters for Decimation and Interpolation*, IEEE Transactions on Acoustics, Speech, and Signal Processing, Vol. ASSP-29, No. 2, April 1981. The general concept of a CIC filter is the low-pass response that results from filtering an input signal with a cascade of N unit-amplitude, rectangular windows of length $R*M$. The system response of such filter is:

$$H(z) = \left[\sum_{k=0}^{R*M-1} z^{-k} \right]^N$$

or

$$H(z) = \frac{(1 - z^{-R*M})^N}{(1 - z^{-1})^N}$$

where

N is the number of CIC stages

R is the rate change (decimation or interpolation)

M is the differential delay in the comb section stages of the filter

The implementation of this filter response with a clever combination of comb filter sections, integrator sections, and up-sampling (for interpolation) and down-sampling (for decimation) gives rise to the hardware-efficient implementation of CIC filters.

Frequency Response Characteristics

The frequency response of a CIC filter is obtained by evaluating the equation in [General Design Guidelines](#) at:

$$z = e^{2j\pi f}$$

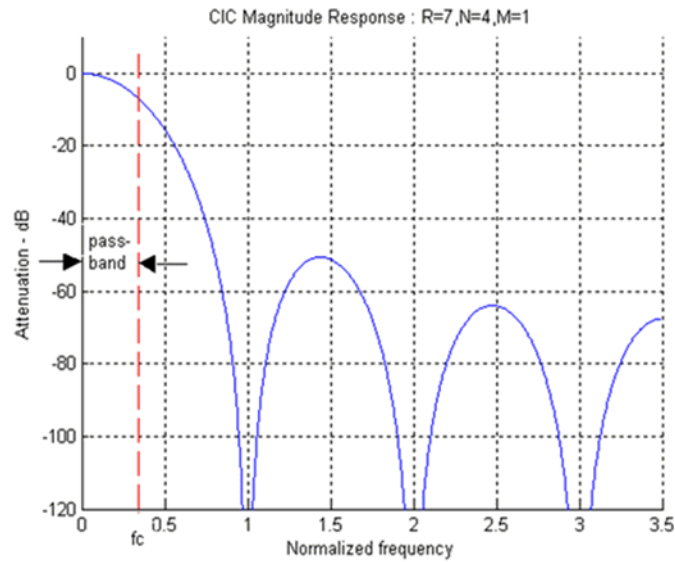
where f is the discrete-time frequency, normalized to the higher frequency in a rate-changing filter — input sampling frequency in a CIC decimation, or output sampling frequency in a CIC interpolator. Evaluating the previous equation in the z -plane at those sample points defined by the preceding equation, gives a magnitude frequency response as shown in the following equation.

$$H(f) = \left(\frac{\sin(\pi RMf)}{\sin(\pi f)} \right)^N$$

This magnitude response is low-pass. In the design process of a CIC filter implementation, the parameters R , M , and N are selected to provide adequate passband characteristics over the frequency range from zero to a predetermined cutoff frequency f_c . The following figure shows the frequency response of a 3-stage ($N = 3$) CIC filter with unity differential delay ($M = 1$) and a sample rate change $R = 7$.

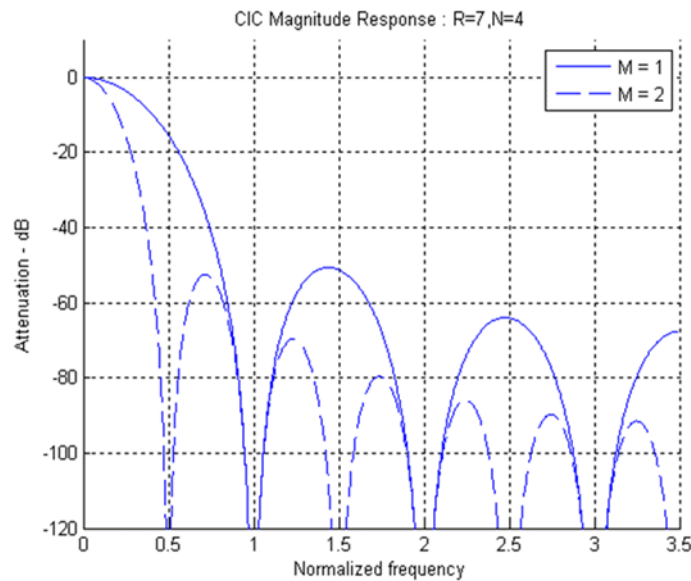
According to the preceding equation and as seen in the following figure, there are nulls in the magnitude response (transfer function zeros) at integer multiples of $f = 1/(RM)$. Thus, the differential delay parameter M can be used as a design parameter to control the placement of the nulls.

Figure 2: CIC Magnitude Response



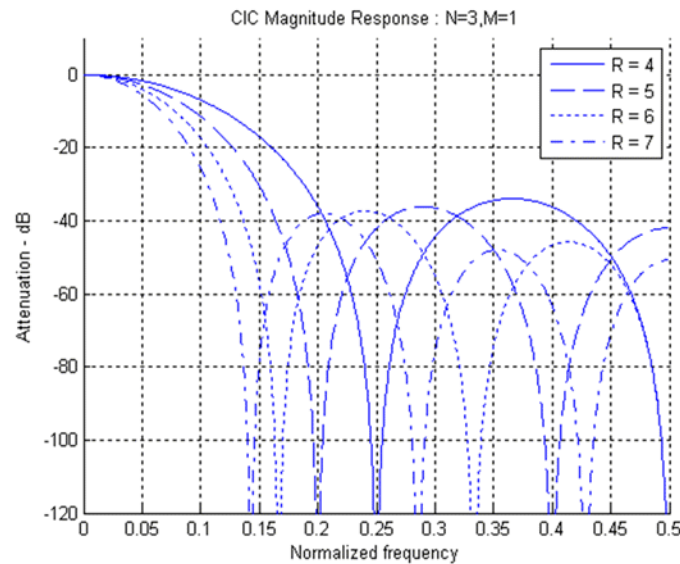
The following figure shows the effect of the differential delay M on the magnitude response of a filter with three stages ($N = 3$) and a sample rate change $R = 7$. Besides the effect on the placement of the response nulls, increasing M also increases the amount of attenuation in the side lobes of the magnitude response.

Figure 3: CIC Magnitude Response Effect of Differential Delay M



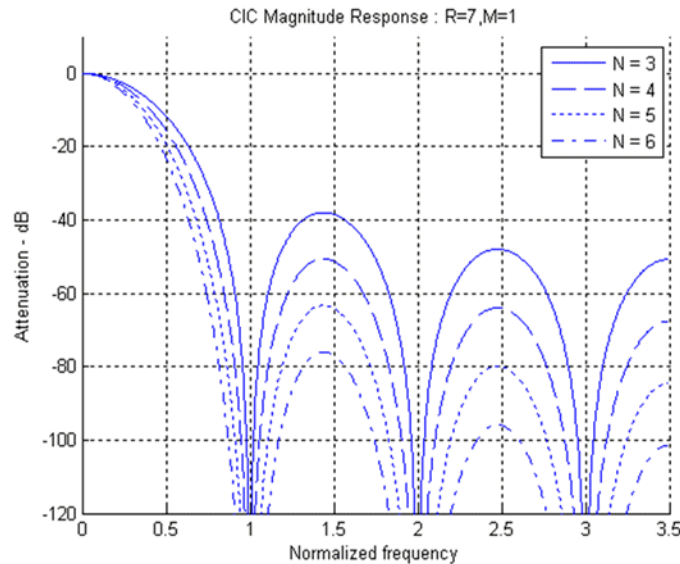
The rate change parameter R can also be used to control the frequency response of the CIC filter. The effect of R on the magnitude response can be seen in the following figure. In essence, increasing the rate change increases the length of the cascaded unit-amplitude, rectangular window of length $R \cdot M$. This results in an increase in attenuation and a decrease in the width of the response side lobes.

Figure 4: CIC Magnitude Response Effect of Rate Change R



The number of stages parameter N can also be used to affect the CIC filter magnitude response. This effect can be understood from the fundamental concept of a cascade of N filtering stages, each with an impulse response of a unit-amplitude, rectangular window. The larger the number of cascaded stages, the more attenuated the magnitude response side lobes become. This can be seen in the following figure.

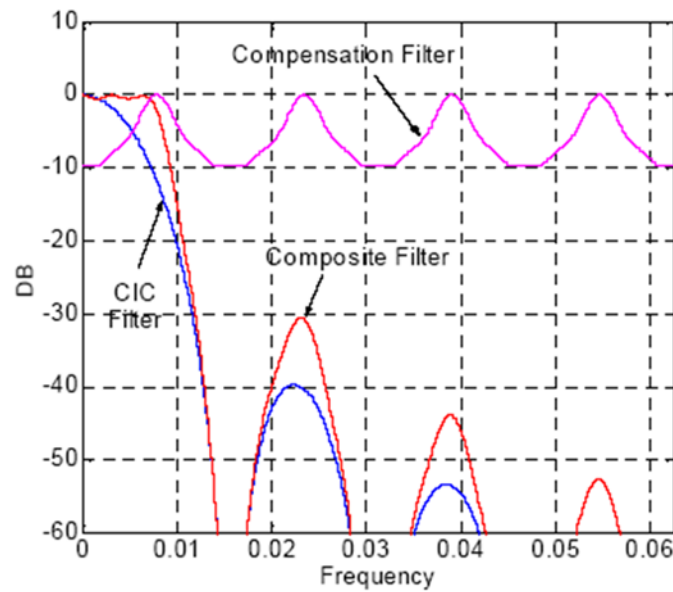
Figure 5: CIC Magnitude Response Effect of Number of Stages N



Increasing N has the effect of increasing the order of the zeros in the frequency response, which in turn increases the attenuation at frequencies in the locality of the zero. This effect is clearly shown in the preceding figure where there is increasing attenuation of the filter side lobes as N is increased.

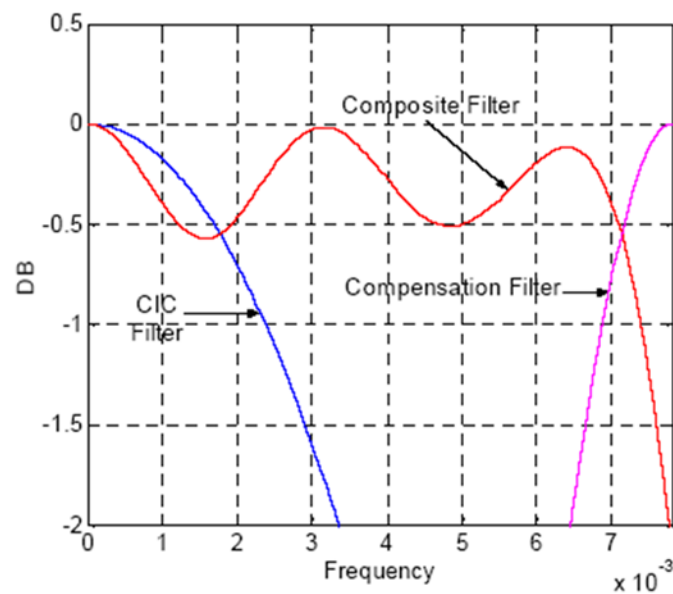
As the order of the zeros increases, the passband droop also increases, thus narrowing the filter bandwidth. The droop is frequently corrected using an additional (non-CIC-based) stage of filtering after the CIC decimator. In the case of a CIC interpolator, the signal can be precompensated using a compensation filter (not part of the CIC Compiler) to flatten the passband frequency response. For a CIC decimator, the compensation filter operates at the decimated sample rate and provides $(x/\sin(x))^N$ shaping. An example of a third-order ($N = 3$) $R = 64$ compensated CIC system is shown in the following figure. The plot shows the uncompensated CIC frequency response, the compensation filter frequency response, and the compensated CIC. In this case, because the number of CIC stages is three, the compensation filter has a cubic response of the form $(x/\sin(x))^3$.

Figure 6: CIC Droop Compensation



The compensation filter coefficients employed were [1, 4, 16, 32, 64, 136, 352, 1312, 352, 136, 64, 32, 16, 4, 1]. The following figure provides an exploded view of the compensated filter passband.

Figure 7: CIC Droop Compensation Exploded View



CIC Decimator

When the output of the filter given by the equation in [General Design Guidelines](#) is decimated (down-sampled) by a factor R , the response of the filter referenced to the lower, down-sampled output rate is expressed in the following equation as:

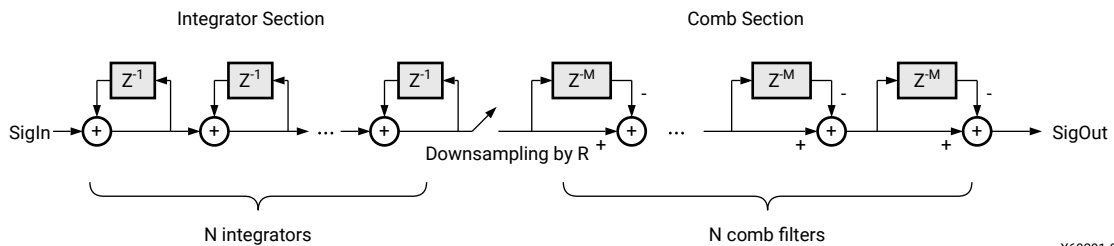
$$H(z) = \left(\frac{1 - z^{-M}}{1 - z^{-1}} \right)^N$$

This response can be viewed as a cascade of N integrators and N comb filters.

$$H(z) = \frac{1}{(1 - z^{-1})^N} (1 - z^{-M})^N$$

The following figure shows a block diagram of a realization of this response. There are two sections to the CIC decimator filter: an integrator section with N integrator stages that processes input data samples at a sampling rate f_s , and a comb section that operates at the lower sampling rate f_s/R . This comb section consists of N comb stages with a differential delay of M samples per stage. The down-sampling operation decimates the output of the integrator section by passing only every R th sample to the comb section of the filter.

Figure 8: CIC Decimation Filter



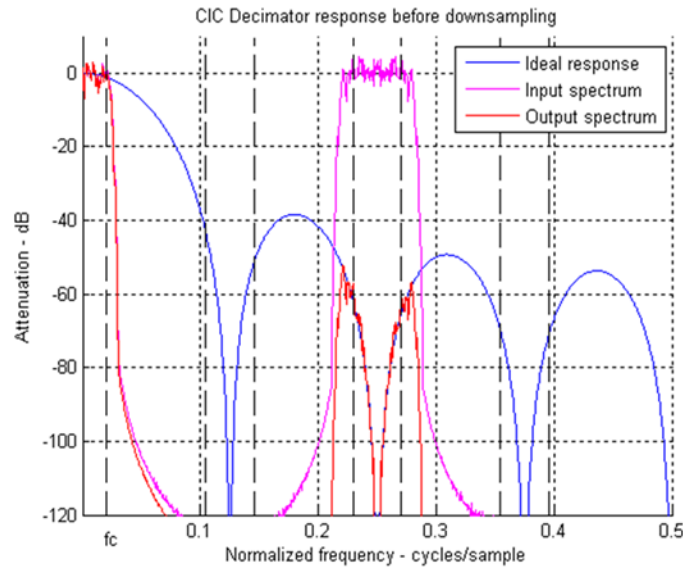
X60291-061126

$$\frac{k}{RM} \pm f_c, \quad k = 1, 2, \dots, \left\lfloor \frac{R}{2} \right\rfloor$$

Referring back to [Figure 2: CIC Magnitude Response](#), when the CIC filter is employed as a decimator, certain frequency bands alias back into the filter passband. Care must be taken to ensure that the integrated side lobe levels do not impact the intended application. The following figure shows an example of a CIC decimator response before down-sampling to show the effect of aliasing. In the following figure, the ideal response of a decimator with sampling rate change of $R = 8$, number of stages $N = 3$, and differential delay $M = 1$ is shown. The

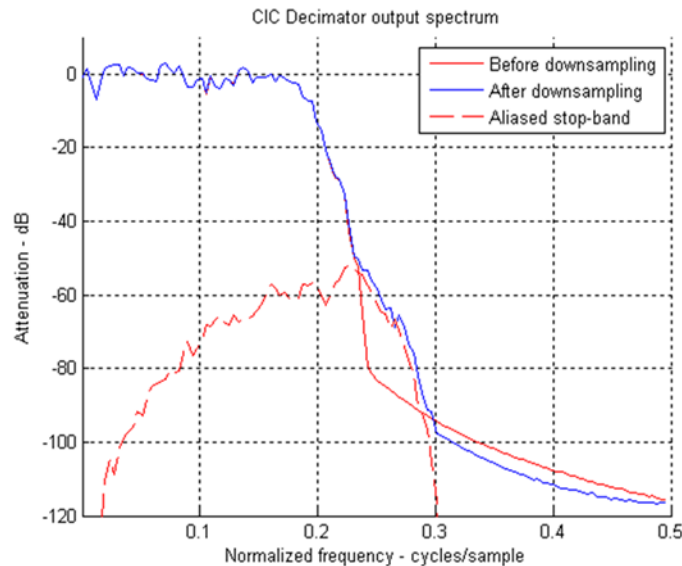
spectrum of the decimator input is also shown containing energy in the intended passband (low frequencies up to a cutoff frequency $f_c = 1/32$ cycles/sample) and in the stopband (around $1/4$ cycles/sample). The output of the decimator (without down-sampling) is shown to demonstrate the attenuation produced by this CIC filter. The dashed vertical lines in the following figure indicate the frequency ranges that alias to the passband when down-sampling. In this figure, the frequency axis is normalized to the (higher) sampling frequency before down-sampling.

Figure 9: CIC Decimator Response Before Down-sampling



The following figure shows the output spectrum of the CIC decimator example. In the following figure, the frequency axis is normalized relative to the lower sampling rate obtained after down-sampling. Because of this re-normalization of frequencies, the plots in the following figure can be conceptualized as a zoomed view of the frequency range from 0 to $1/(2 \cdot R) = 1/16$ cycles/sample of the preceding figure.

Figure 10: CIC Decimator Output Spectrum



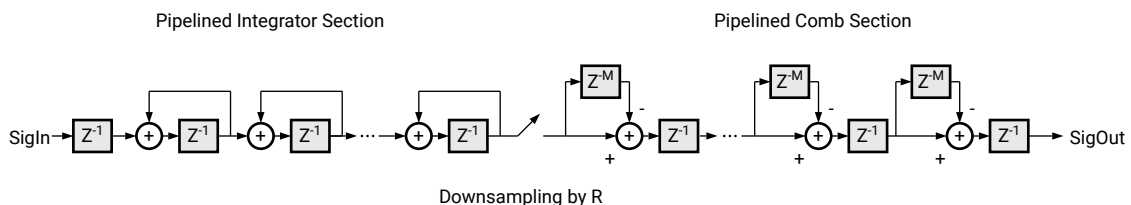
- The solid red plot shows the CIC output spectrum if no aliasing occurred.
- The dashed red plot shows the stopband output spectrum when aliased due to down-sampling. This aliased spectrum affects the final output of the CIC decimator by contributing additively to the output spectrum.
- The solid blue plot is the actual output of the CIC decimator, which clearly shows the contribution of the aliased spectrum from down-sampling.

Again, care must be taken to ensure that the CIC decimator parameters are properly chosen to avoid detrimental effects from aliasing.

Pipelined CIC Decimator

To support high system clock frequencies, the CIC decimator is implemented using the pipelined architecture shown in the following figure.

Figure 11: Pipelined CIC Decimator



X60293-061126

The CIC datapath undergoes internal register growth that is a function of all the design parameters: N , M , and R , in addition to the input sample precision B . As shown in *An Economical Class of Digital Filters for Decimation and Interpolation*, the output bit width of a CIC decimator with full precision is given by:

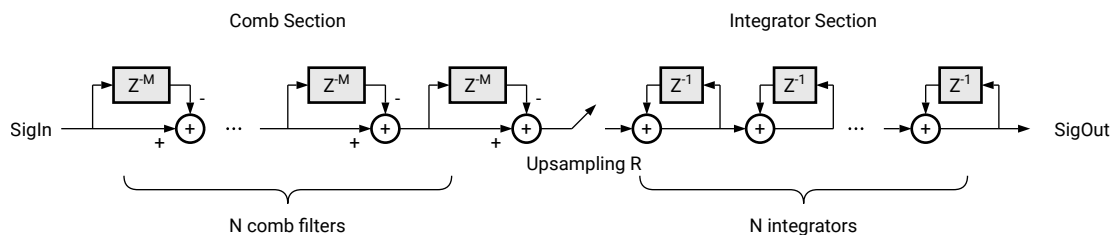
$$B_{\max} = \lceil N \log_2(RM) + B \rceil$$

where $\lceil \cdot \rceil$ indicates the ceiling operator. The CIC Compiler supports both full and limited precision output. For full precision, the CIC decimator implementation uses $B_{\max}$ bits internally for each of the integrator and differentiator stages. This introduces no quantization error at the output. For limited precision (that is, output bit width less than $B_{\max}$), the registers in the integrator and comb stages are sized to limit the quantization noise variance at the output. Consequently, the hardware resources in a CIC decimator implementation can be reduced when using limited precision output at the cost of quantization noise. This ability to trade off resources and quantization noise is important to achieve an optimum implementation.

CIC Interpolator

The structure for a CIC interpolator filter is shown in the following figure. This structure is similar to that of a CIC decimator with the integrator and comb sections in reverse order. In this case, there is an up-sampling of data by a factor R between the comb and integrator sections. This rate increase is done by inserting $R-1$ zero-valued samples between consecutive samples of the comb section output. The up-sampled and filtered data stream is output at the sample rate f_s .

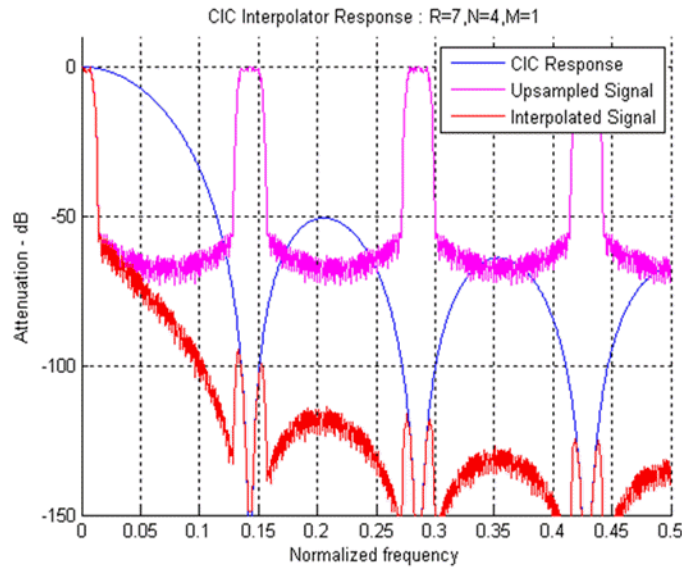
Figure 12: CIC Interpolator



X60292-061126

For interpolation, the response of the CIC filter is applied to the up-sampled (zero-valued samples inserted) input signal. The effect of this processing is shown in the following figure in a filter with rate change $R = 7$, number of stages $N = 4$, and differential delay $M = 1$. The peaks in the output interpolated signal show the effect of the magnitude response of the CIC filter applied to the spectrum images of the up-sampled input signal.

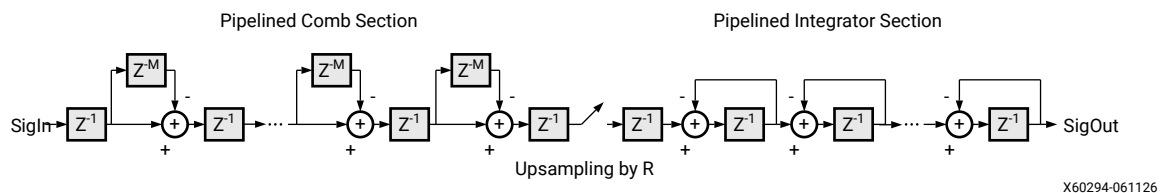
Figure 13: CIC Interpolator Response



Pipelined CIC Interpolator

Similarly to the CIC decimator, the CIC interpolator core implementation uses a pipelined structure to support high system clock frequencies. The following figure shows this pipelined structure.

Figure 14: Pipelined CIC Interpolator



Register Growth in CIC Interpolator

The datapath in a CIC interpolator also undergoes internal register growth that is a function of all the design parameters: N , M , and R , in addition to the input sample precision B . As shown in *An Economical Class of Digital Filters for Decimation and Interpolation*, the registers in the comb and integrator sections grow monotonically, with the maximum register size occurring at the output of the last stage (output of the CIC filter). The maximum register width is given by the following equation:

$$B_{\max} = \left\lceil \log_2 \left(\frac{(RM)^N}{R} \right) + B \right\rceil$$

where $\lceil \cdot \rceil$ indicates the ceiling operator. The CIC Compiler always sizes the internal stage registers according to the register growth as described in *An Economical Class of Digital Filters for Decimation and Interpolation*. The output of the filter can be selected to be full or limited precision (with truncation or rounding) to accommodate an output width specific to an application. Using limited precision does not affect the internal register sizes; only the final stage output is scaled, and rounded if required, to provide the selected output width.

Output Width and Gain

As shown by the equations in [Pipelined CIC Decimator](#) and [Register Growth in CIC Interpolator](#), the gain of a CIC filter is a function of all the key design parameters. When the output width is equal to the maximum register width, the core outputs the full precision result and the magnitude of the core output reflects the filter gain. When the output width is set to less than the maximum register width, the output is truncated with a corresponding reduction in gain.

When the core is configured to have a programmable rate change, there is a corresponding change in gain as the filter rate is changed. When the output is specified to full precision, the change in gain is apparent in the core output magnitude as the rate is changed. When the output is truncated, the core shifts the internal result, given the $B_{\max}$ for the current rate change, to fully occupy the output bits.

Clocking

The core uses a single clock for all functions and interfaces.

Resets

The core uses a single active-Low reset signal for all functions and interfaces.

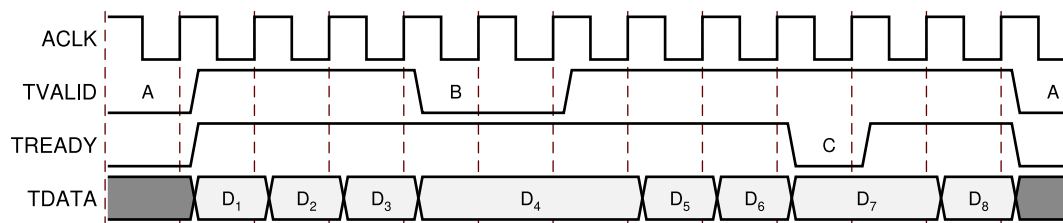
Protocol Description

Control Signals and Timing

Symbol data to be processed is loaded into the CIC Compiler core using the Data Input Channel. Processed symbol data is unloaded using the Data Output Channel. Both of these use the AXI4-Stream protocol. The following figure shows the basics of this protocol.

`TVALID` is driven by the channel master to show that it has data to transfer, and `TREADY` is driven by the channel slave to show that it is ready to accept data. When both `TVALID` and `TREADY` are High, a transfer takes place. Points A in the diagram show clock cycles where no data is transferred because neither the master or the slave is ready. Point B shows two clock cycles where data is not transferred because the Master does not have any data to transfer. This is known as a master waitstate. Point C shows a clock cycle where no data is transferred because the slave is not ready to accept data. This is known as a slave waitstate. Master and slave waitstates can extend for any number of clock cycles.

Figure 15: AXI Transfers and Terminology



When the master asserts `TVALID` High, it must remain asserted (and the associated data remain stable) until the slave asserts `TREADY` High.

The preceding figure shows the loading of 8 samples. The upstream master drives `TVALID` and the CIC Compiler drives `TREADY`. In this case, both the master and the CIC Compiler insert waitstates.

The preceding figure also shows the unloading of 8 samples. The CIC Compiler drives `TVALID` and the downstream slave drives `TREADY`. In this case, both the CIC Compiler and the slave insert waitstates. This only applies when the core is configured to have a `TREADY` port on the Data Output Channel (`XCO HAS_DOUT_TREADY = TRUE`). When this is false, there is no `TREADY` signal on the Data Output Channel and the downstream slave cannot insert waitstates. The slave must be able to respond immediately on every clock cycle where the CIC Compiler produces data (`m_axis_data_tvalid` asserted High). If the slave cannot respond immediately, then data is lost.

For multiple-channel implementations, the CIC Compiler core supports time-multiplexed input and output. The filter input data in the `DATA` field of the Data Input Channel `TDATA` vector (`s_axis_data_tdata`) is expected to have an ordered, time-multiplexed format. The core produces time-multiplexed output data on the `DATA` field of the Data Output Channel `TDATA` vector (`m_axis_data_tdata`). Two additional fields are included in multichannel implementation. The `CHAN_SYNC` field in the Data Output Channel `TUSER` vector (`m_axis_data_tuser`) indicates the output corresponding to the first channel in the time-multiplexed stream. The `CHAN_OUT` field in the Data Output Channel `TUSER` vector (`m_axis_data_tuser`) contains the channel number for each output in the time-multiplexed stream.

For programmable rate implementations, the `RATE` field in the Configuration Channel `TDATA` vector (`s_axis_config_tdata`) controls the rate change in the CIC Compiler filter core. The `RATE` field is sampled when `s_axis_config_tvalid` and `s_axis_config_tready` are both asserted High. The core uses the new `RATE` value on the next input sample, for a single channel implementation, or the next input to the first channel, for multiple channel implementations.

All of the waveforms are shown with `HAS_DOUT_TREADY = FALSE`. Setting this to `TRUE` allows the downstream data slave to delay the data output of the CIC Compiler. It also allows the Data Input Channel to buffer samples so that they can be supplied at a faster rate than the core can process them.

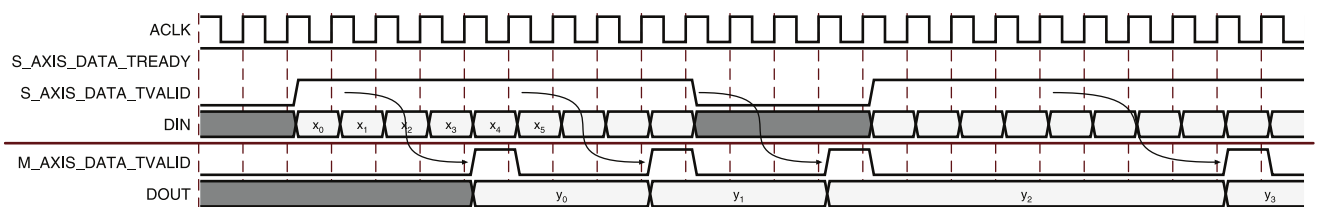
To simplify the waveforms, the following field aliases are used

- `DIN` is used to represent the `DATA` field in the Data Input Channel `TDATA` vector (`s_axis_data_tdata`)
- `DOUT` is used to represent the `DATA` field in the Data Output Channel `TDATA` vector (`m_axis_data_tdata`)
- `CHAN_SYNC` is used to represent the `CHAN_SYNC` field in the Data Output Channel `TUSER` vector (`m_axis_data_tuser`)
- `CHAN_OUT` is used to represent the `CHAN_OUT` field in the Data Output Channel `TUSER` vector (`m_axis_data_tuser`)
- `RATE` is used to represent the `RATE` field in the Configuration Channel `TDATA` vector (`s_axis_config_tdata`)

Decimator

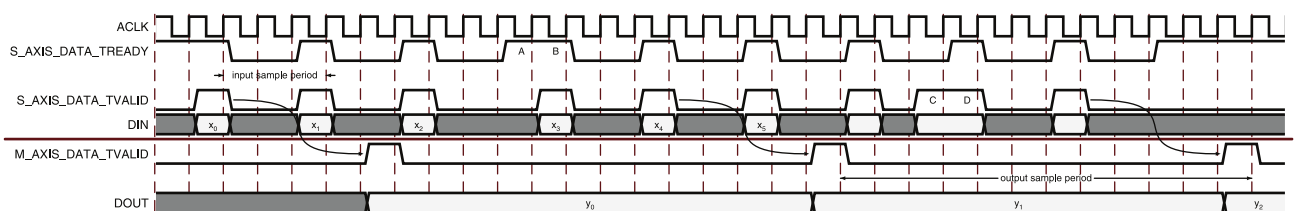
The timing for a CIC decimator with a down-sampling factor $R = 4$ is shown in the following figure. In this example, the core is not oversampled and can accept a new input sample on every clock edge. Some number of clock cycles after the first input sample has been written to the filter, `m_axis_data_tvalid` is asserted by the filter to indicate that the first output sample is available. This time interval is a function of the down-sampling factor R and a fixed latency that is related to internal pipeline registers in the core. The number of pipeline stages depends on the core parameters. After the first output sample has been produced, subsequent outputs are available every R clock cycles.

Figure 16: CIC Decimator - Fixed Rate, Single Channel



The following figure shows the timing for the same filter configuration with an input sample period of 3. At point A in the waveform, the CIC Compiler is ready to accept data but the master does not provide it. The CIC Compiler continues to ask until it is provided (point B). At point C in the waveform, the master provides data before the CIC Compiler requests it. The master has to continue supplying this data until the CIC Compiler accepts it (at point D).

Figure 17: CIC Decimator - Fixed Rate, Single Channel, Oversampled



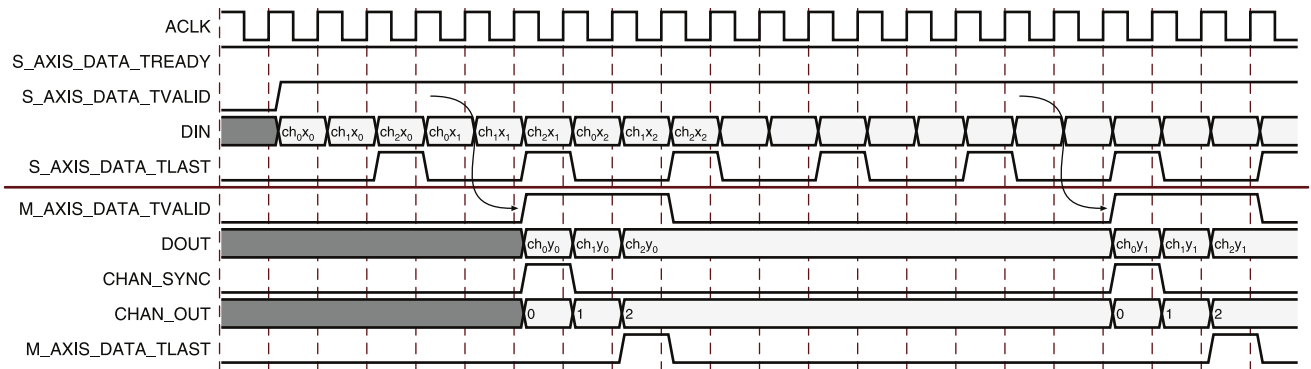
Multichannel Decimators can be configured to produce data in two timing modes, Block and Streaming:

- Block mode: samples for the channels are produced back-to-back. That is, the data for channel $N+1$ is produced immediately after the data for channel N .
- Streaming mode: samples for the channels are produced evenly over the entire sample period.

These modes operate independently of the AXI4 interface and they refer to the part of the core that processes the data. When `HAS_DOUT_TREADY = 1` the AXI4 interface can buffer data in the Data Output Channel which means that streaming mode can start to behave like block mode. If the downstream system does not consume data when it first becomes available, the Data Output Channel can start to fill. In this case, the Data Output Channel produces back-to-back data (using `m_axis_data_tvalid`) until the buffer in the channel is empty, even though the processing part of the core did not produce it back-to-back.

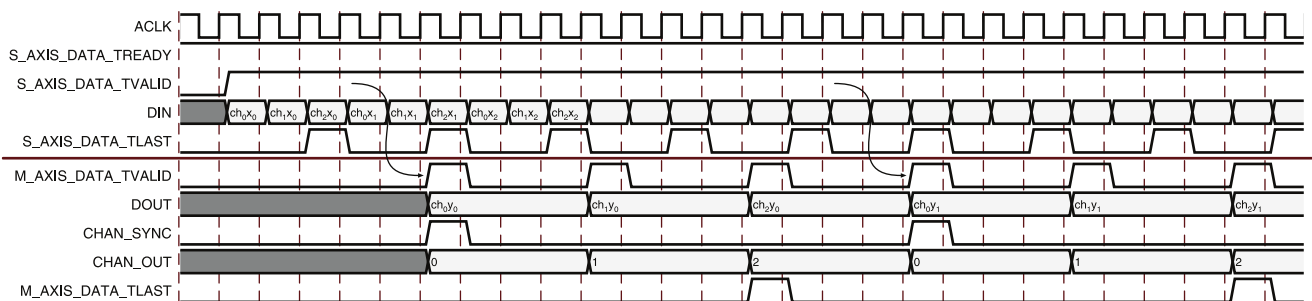
The following figure shows the timing for a multichannel CIC decimator with a rate change $R = 4$. In this example the decimator filter handles three channels of data and is configured to use the block-based interface. The input to the decimator `DIN` shows the time-multiplexed samples with labels to indicate the corresponding channel number. The output of the decimator `DOUT` shows the time-multiplexed data.

Figure 18: CIC Decimator - Fixed Rate, Multichannel, Block interface



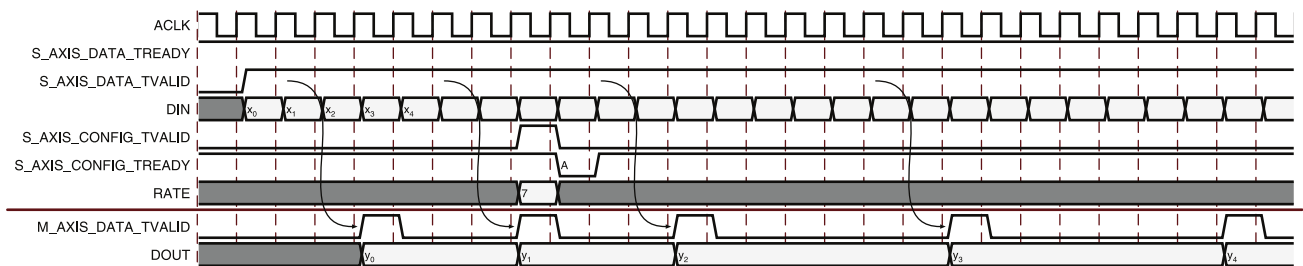
The following figure shows the timing for the same filter configuration using the streaming interface.

Figure 19: CIC Decimator - Fixed Rate, Multichannel, Streaming interface



The following figure shows the timing for a CIC decimator with programmable rate. In the timing diagram, the decimator is shown with an initial down-sampling rate value of 4. After some time, the down sampling rate is changed to 7 by setting the value in the RATE field to 7 and asserting `s_axis_config_tvalid` at point A in the waveform. As the rate is only applied when the next sample is accepted by the CIC Compiler, `s_axis_config_tready` deasserts for a cycle while the rate change is applied. This prevents the upstream master providing a new rate which cannot be accepted by the core.

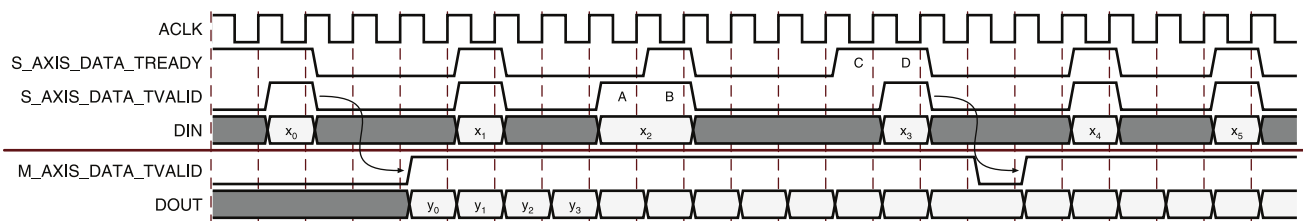
Figure 20: CIC Decimator with Programmable Rate



Interpolator

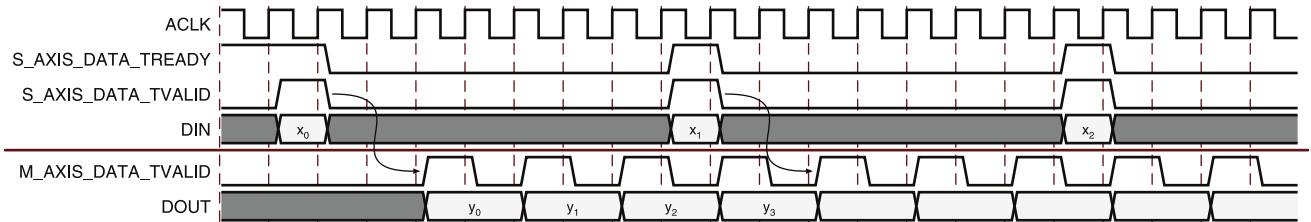
The following figure shows the timing for a CIC interpolator with an up-sampling factor $R = 4$. A new input sample can be accepted by the core every fourth cycle of the clock. After the initial start-up latency, `m_axis_data_tvalid` is asserted, and a new filter output is available on every subsequent clock edge. For every input delivered to the filter core, four output samples are generated. At point A in the waveform, the master provides data before the CIC Compiler requests it. The master has to continue supplying this data until the CIC Compiler accepts it (at point B). At point C in the waveform, the CIC Compiler is ready to accept data but the master does not provide it. The CIC Compiler continues to ask until it is provided (point D).

Figure 21: CIC Interpolator - Fixed Rate, Single Channel



The following figure shows the same filter configuration with an input sample period of 8.

Figure 22: CIC Interpolator - Fixed Rate, Single Channel, Oversampled



Multichannel Interpolators can be configured to consume data in two timing modes, Block and Streaming:

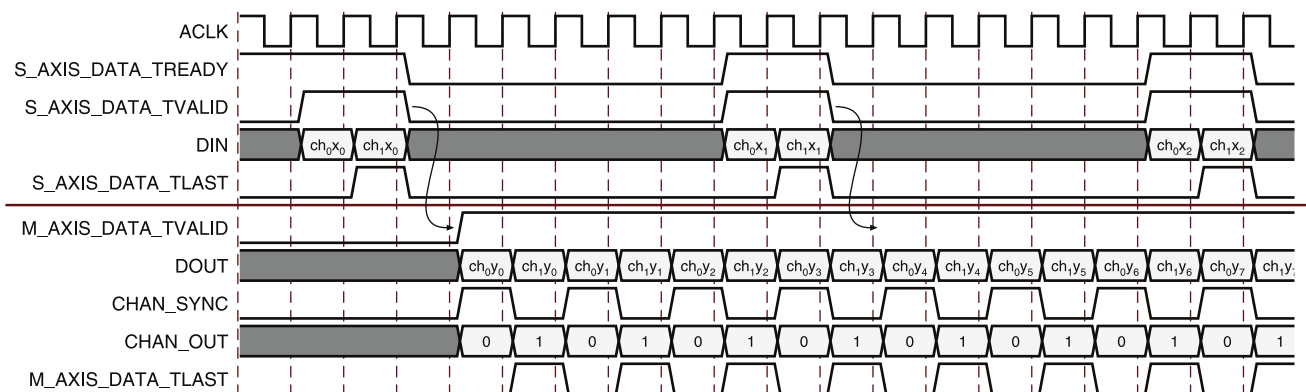
- Block mode: samples for the channels are consumed back-to-back. That is, the data for channel N+1 is consumed immediately after the data for channel N.
- Streaming mode: samples for the channels are consumed evenly over the entire sample period.

Refer to the following figures for more information.

These modes operate independently of the AXI4 interface and they refer to the part of the core that processes the data. When `HAS_DOUT_TREADY = 1` the AXI4 interface can buffer data in the Data Input Channel which means that streaming mode might start to behave like block mode. Until its buffer is full, the Data Input Channel requests back-to-back data (using `s_axis_data_tready`) even though the processing part of the core does not consume it immediately.

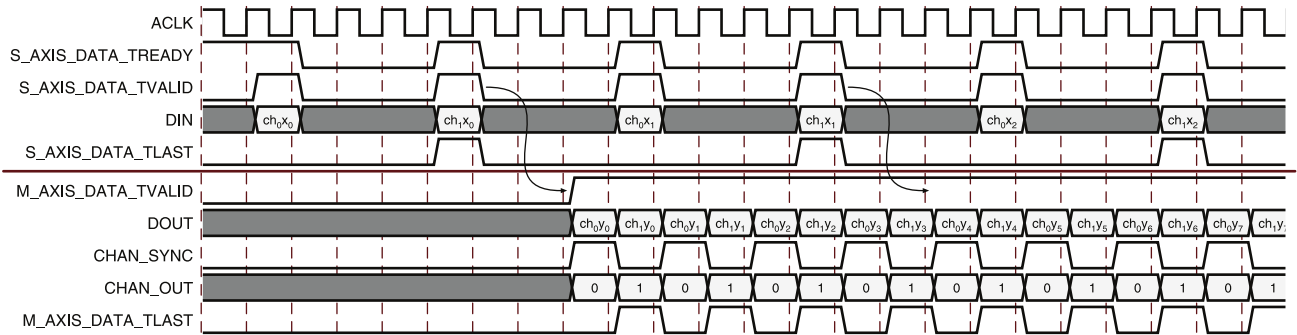
The following figure shows the timing for a multichannel CIC interpolator with a rate change $R = 4$. In this example the interpolator filter handles two channels of data and uses the block-based interface. The input DIN shows the time-multiplexed samples with labels to indicate the corresponding channel number. The output DOUT shows the time-multiplexed data samples.

Figure 23: CIC Interpolator - Fixed Rate, Multichannel, Block interface



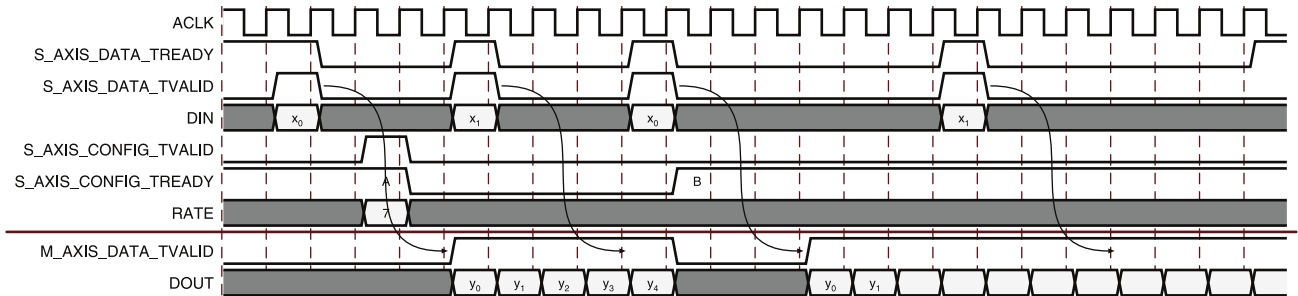
The following figure shows the same filter configuration using the streaming interface.

Figure 24: CIC Interpolator - Fixed Rate, Multichannel, Streaming interface



The following figure shows the timing for a CIC interpolator with programmable rate. In the timing diagram, the interpolator is shown with an initial up-sampling rate value of 4. After some time, the up-sampling rate is changed to 7 by setting the RATE field to 7 and asserting `s_axis_config_tvalid` at point A in the waveform. As the rate is only applied when the next sample for the first channel is accepted by the CIC Compiler, `s_axis_config_tready` deasserts until the rate change is applied (point B). In this example, the sample (x_1) following point A is for the second channel, so the rate is not applied here. This prevents the upstream master providing a new rate which cannot be accepted by the core.

Figure 25: CIC Interpolator with Programmable Rate



Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard AMD Vivado™ design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
- *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
- *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
- *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))

Customizing and Generating the Core

This section includes information about using AMD tools to customize and generate the core in the AMD Vivado™ Design Suite.

If you are customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#)) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)) and the *Vivado Design Suite User Guide: Getting Started* ([UG910](#)).

Figures in this chapter are illustrations of the Vivado IDE. The layout depicted here might vary from the current version.

Core Parameters

The CIC Compiler core customization screen in the Vivado IDE has three pages used to configure the core plus three informational/analysis tabs.



IMPORTANT! *The component name at the top of the GUI is the name of the core component to be instantiated. The name must begin with a letter and be composed of the following characters: a to z, 0 to 9, and “_”.*

IP Symbol Tab

The IP Symbol tab illustrates the core pinout.

Frequency Response Tab

The Freq. Response tab displays the filter frequency response (magnitude only).



IMPORTANT! *The frequency axis in this plot is normalized frequency.*

Although the values in the Vivado IDE plot range from 0 to 1.0, they represent the range from 0 to 1/2 the sampling frequency. The normalizing sampling frequency implied in the plot depends on the type of filter. For a CIC decimator, the normalizing sampling frequency is the higher input sampling frequency. For a CIC interpolator, the normalizing frequency is the higher output sampling frequency.

- **Response Magnitude:** Specifies the magnitude scaling of the frequency response: Normalized; Full Precision (the absolute filter gain) and Output Quantization (the effective filter gain given the core output width). In previous versions of the core, the frequency response was always normalized. All plots shown in [General Design Guidelines](#) use normalized magnitude.
- **Passband Range:** Two fields are available to specify the passband range, the left-most being the minimum value and the right-most the maximum value. The values are specified in the same units as on the graph x-axis (for example, normalized to pi radians per second). For the specified range, the passband maximum, minimum and ripple values are calculated and displayed (in dB).
- **Stopband Range:** Two fields are available to specify the stopband range, the left-most being the minimum value and the right-most the maximum value. The values are specified in the same units as on the graph x-axis (for example, normalized to pi radians per second). For the specified range, the stopband maximum value is calculated and displayed (in dB).

Note: Any range can be specified for the passband or stopband, allowing closer analysis of any region of the response.

Implementation Details Tab

- **Resource Estimates:** Based on the options selected, this field displays the DSP48E1 slice count and 18K block RAM numbers. The resource numbers are an estimate; for exact resource usage, and slice/ LUT-FlipFlop pair information, consult a MAP report.
- **AXI4-Stream Port Structure:** This section shows how the CIC Compiler fields are mapped to the AXI4 channels.

Filter Options

- **Filter Type:** The CIC Compiler core supports both interpolator and decimator types. When the filter type is selected as decimator, the input sample stream is down-sampled by the factor R. When an interpolator is selected, the input sample is up-sampled by R.
- **Number of Stages:** Number of integrator and comb stages. If N stages are specified, there are N integrators and N comb stages in the filter. The valid range for this parameter is 3 to 6.
- **Differential Delay:** Number of unit delays employed in each comb filter in the comb section of either a decimator or interpolator. The valid range of this parameter is 1 or 2.
- **Number of Channels:** Number of channels to support in implementation. Valid values for this parameter are 1 to 16, 32, 64 and 128.
- **Fixed/Programmable:** Type of rate change is fixed or programmable.
- **Fixed or Initial Rate:** Rate change factor (for fixed type) or initial rate change factor (for programmable type). For an interpolation filter, the rate change specifies the amount of up-sampling. For a decimator, it specifies the amount of down-sampling.
- **Minimum Rate:** Minimum rate change factor for programmable rate change.
- **Maximum Rate:** Maximum rate change factor for programmable rate change.
- **Hardware Oversampling Specification Format:** Selects which format is used to specify the hardware oversampling rate, the number of clock cycles available to the core to process an input sample and generate an output. This value directly affects the level of parallelism in the core implementation and resources used. When Frequency Specification is selected, you need to specify the Input Sampling Frequency and Clock Frequency. The ratio between these values along with other core parameters determine the hardware oversampling rate. When Sample Period is selected, you need to specify the integer number of clock cycles between input samples.
- **Input Sample Frequency:** This field can be an integer or real value. It specifies the sample frequency for one channel. The upper limit is set based on the clock frequency and filter parameters such as interpolation rate and number of channels.
- **Clock Frequency:** This field can be an integer or real value. The limits are set based on the sample frequency, interpolation rate and number of channels.

Note: This field influences architecture choices only; the specified clock rate might not be achievable by the final implementation.

- **Input Sample Period:** Integer number of clock cycles between input samples. When you specify multiple channels, this value needs to be the integer number of clock cycles between the time division multiplexed input sample data stream.

Implementation Options

- **Input Data Width:** Number of bits for input data. The valid range of this parameter is 2 to 32. In IP integrator, this parameter is auto-updated.
- **Quantization:** Type of quantization for limited precision output, Full Precision or Truncation. This quantization applies only to the output and is not applied in the intermediate stages of the CIC Compiler filter.
- **Output Data Width:** Number of bits for output data. The valid range of this parameter is up to 48 bits with the minimum value set to the input data width.
- **Use Xtreme DSP Slice:** Use DSP hardware primitive slices in the filter implementation.
- **Use Streaming Interface:** Specifies if a streaming interface is used for multiple channel implementations. Refer to [Decimator](#), for further details.
- **Has DOUT TREADY:** Specifies if the Data Output Channel has a `TREADY` signal.
- **ACLKEN:** Determines if the core has a clock enable input (`aclken`).
- **ARESETN:** Determines if the core has an active-Low synchronous clear input (`aresetn`).

Note:

- The signal `aresetn` always takes priority over `aclken`, that is, `aresetn` takes effect regardless of the state of `aclken`.
- The signal `aresetn` is active-Low.
- The signal `aresetn` needs to be held active for at least two clock cycles. This is because, for performance, `aresetn` is internally registered before being fed to the reset port of primitives.

Summary Tab

In addition to all the parameterization values of the core, the summary page displays:

- **Bits per Stage:** The number of bits used in each of the stages of the CIC Compiler filter implementation. These numbers are computed based on the register growth analysis presented in *An Economical Class of Digital Filters for Decimation and Interpolation*.

- **Latency:** The input to output latency in the CIC Compiler core implementation. When HAS_DOUT_TREADY is TRUE then the actual latency might be greater than reported because throughput can be controlled by the system connected to the Data Output Channel. The value reported by the Vivado Design Suite is the minimum latency.

User Parameters

The following table shows the relationship between the fields in the AMD Vivado™ IDE and the user parameters (which can be viewed in the Tcl Console).

Table 1: User Parameters

Vivado IDE Parameter/Value	User Parameter/Value	Default Value
Filter Type	filter_type	Interpolation
Number of Stages	number_of_stages	3
Differential Delay	differential_delay	1
Number of Channels	number_of_channels	1
Sample Rate	sample_rate_changes	Fixed
Fixed Or Initial Rate	fixed_or_initial_rate	4
Minimum Rate	minimum_rate	4
Maximum Rate	maximum_rate	4
Rate Specification	ratespecification	Frequency_Specification
Input Sample Frequency	input_sample_frequency	0.001
Clock Frequency	clock_frequency	200.0
Sample Period	sampleperiod	4
Input Data Width	input_data_width	18
Quantization	quantization	Full_precision
Output Data Width	output_data_width	22
Use DSP48 Slice	use_xtreme_dsp_slice	True
ACLKEN	has_aclken	False
ARESETn	has_aresetn	False
Output Tready	has_dout_tready	False

System Generator for DSP GUI

The CIC Compiler core is available through AMD System Generator for DSP, a design tool that enables the use of the model-based design environment, Simulink® product for FPGA design. The CIC Compiler core is one of the DSP building blocks provided in the AMD blockset for Simulink®. The core can be found in the AMD Blockset in the DSP section. The block is called CIC Compiler v4.0. Refer to the System Generator User Manual for more information.

This section describes each tab of the System Generator for DSP GUI and details the parameters that differ from the Vivado IDE. Refer to [Core Parameters](#) for detailed information about all other parameters.

Filter Specification Tab

The Filter Specification tab is used to define the basic filter configuration as in [Filter Options](#) in the Vivado IDE.

- **Hardware Oversampling Specification Format:** Selects which method is used to specify the hardware oversampling rate. This value directly affects the level of parallelism of the core implementation and resources used. When Maximum Possible is selected, the core uses the maximum oversampling given the sample period of the signal connected to the Data field of the `s_axis_data_tdata` port. When Hardware Oversampling Rate is selected, you can specify the oversampling rate. When Sample Period is selected, the core clock is connected to the system clock, and the value specified for the Sample Period parameter sets the input sample rate the core supports. The Sample Period parameter also determines the hardware oversampling rate of the core. When Sample Period is selected, the core is forced to use the `s_axis_data_tvalid` control port. See [Decimator](#), for more details on the core control ports.
- **Sample Period:** Specifies the input sample period supported by the core.
- **Hardware Oversampling Rate:** Specifies the hardware oversampling rate applied to the core.

Implementation Options

The Implementation tab is used to define implementation options. Refer to [Implementation Options](#) in the Vivado IDE for complete parameter details.

- **Has ARESETN:** Specifies whether the core has a reset pin (the equivalent of selecting the Has ARESETN option in the Vivado IDE).
- **Has ACLKEN:** Specifies whether the core has a clock-enable pin (the equivalent of selecting the Has ACLKEN option in the Vivado IDE).
- **Has DOUT TREADY:** Specifies whether the Data Output Channel has a `TREADY` pin (the equivalent of selecting the `HAS_DOUT_TREADY` option in the Vivado IDE).
- **FPGA Area Estimation:** Refer to System Generator documentation for details.

Using the CIC Compiler IP Core

The Vivado IDE performs error-checking on all input parameters. Resource estimation and optimum latency information are also available.

Several files are produced when a core is generated, and customized instantiation templates for Verilog and VHDL design flows are provided in the .veo and .vho files, respectively. For detailed instructions, refer to the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))

XCI Parameters

The following table defines valid entries for the XCI parameters. Parameters are not case sensitive. Default values are displayed in bold. AMD strongly suggests that XCI parameters are not manually edited in the XCI file; instead, use the Vivado IDE to configure the core and perform range and parameter value checking. The XCI parameters are useful for defining the interface to other AMD tools.

Table 2: XCI Parameters

XCI Parameter	Valid Values
Component_Name	ASCII text using characters a-z, 0-9, and underscore; starting with a letter
Filter_Type	Interpolation, Decimation
Number_Of_Stages	3, 4, 5, 6
Differential_Delay	1, 2
Number_Of_Channels	1-16, 32, 64, 128
Sample_Rate_Changes	Fixed, Programmable
Fixed_Or_Initial_Rate	4-8192
Minimum_Rate	4-8191
Maximum_Rate	5-8192
RateSpecification	Frequency_Specification, Sample_Period
Input_Sample_Frequency	0.000001-600.0
Clock_Frequency	0.000001-600.0
SamplePeriod	1-10000000
Input_Data_Width	2-32 (default 18)
Quantization	Full_Precision, Truncation
Output_Data_Width	2-104
Use_Xtreme_DSP_Slice	false, true
Use_Streaming_Interface	true, false
HAS_DOUT_TREADY	false, true
HAS_ACLKEN	false, true
HAS_ARESETN	false, true

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Constraining the Core

Required Constraints

This section is not applicable for this IP core.

Device, Package, and Speed Grade Selections

This section is not applicable for this IP core.

Clock Frequencies

This section is not applicable for this IP core.

Clock Management

This section is not applicable for this IP core.

Clock Placement

This section is not applicable for this IP core.

Banking

This section is not applicable for this IP core.

Transceiver Placement

This section is not applicable for this IP core.

I/O Standard and Placement

This section is not applicable for this IP core.

Simulation

For comprehensive information about AMD Vivado™ simulation components, as well as information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#)).



IMPORTANT! For cores targeting 7 series or Zynq 7000 devices, UNIFAST libraries are not supported. AMD IP is tested and qualified with UniSim libraries only.

Synthesis and Implementation

For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

C Model

The CIC Compiler core bit-accurate C model is a self-contained, linkable, shared library that models the functionality of this core with finite precision arithmetic. This model provides a bit-accurate representation of the various modes of the CIC Compiler core, and is suitable for inclusion in a larger framework for system-level simulation or core-specific verification.

The C model is an output product of the CIC Compiler IP; it is generated together with the core's other output products, and the resulting ZIP files are delivered in the `cmodel` directory of the generated core.

The C model is bit-accurate but not cycle-accurate, performing exactly the same arithmetic operations as the core. It does not model the latency or interface signals of the core.

The CIC Compiler C model supports the following features:

- Interpolation and decimation filters
- 1 to 6 integrator/comb stages
- Differential delay of 1 or 2
- Fixed and programmable sample rate
- 1 to 128 time-division multiplexed (TDM) channels
- Full-precision or truncated output

Unpacking and Model Contents

There are separate ZIP files containing all the files necessary for use with a specific computing platform. Each ZIP file contains:

- The C model shared library
- GNU Multiple Precision Arithmetic (GMP) library
- The C model header file
- The example code showing customers how to call the C model
- Documentation

Note: The C model uses the GMP library, which is provided in the ZIP files. Refer to the *GNU Multiple Precision Arithmetic (GMP) Library* (gmplib.org/) library.

Table 3: C Model Zip File Contents: Linux

File	Description
cic_compiler_v4_0_bitacc_cmodel.h	Header file which defines the C model API
xip_common_bitacc_cmodel.h	Common C model header file, used by the C model
xip_mpz_bitacc_cmodel.h	Arbitrary-precision (mpz) array header file, used by the C model
libIp_cic_compiler_v4_0_bitacc_cmodel.so	Model shared object library
libgmp.so.11	GMP library, used by the C model
libgmpxx.so.4	GMP Class library, used internally by the C model
gmp.h	GMP header file, used by the C model
run_bitacc_cmodel.c	Example program for calling the C model
cic_compiler_v4_0_bitacc_mex.cpp	MATLAB® MEX function source
make_cic_compiler_v4_0_mex.m	MATLAB MEX function compilation script
run_cic_compiler_v4_0_mex.m	MATLAB MEX function example script
@cic_compiler_v4_0_bitacc	MATLAB MEX function class directory

Table 4: C Model Zip File Contents: Windows

File	Description
cic_compiler_v4_0_bitacc_cmodel.h	Header file which defines the C model API
xip_common_bitacc_cmodel.h	Common C model header file, used by the C model
xip_mpz_bitacc_cmodel.h	Arbitrary-precision (mpz) array header file, used by the C model
libIp_cic_compiler_v4_0_bitacc_cmodel.dll	Model dynamically linked library
libIp_cic_compiler_v4_0_bitacc_cmodel.lib	Model .lib file for compiling
libgmp.dll	GMP library, used by the C model
libgmp.lib	GMP .lib file for compiling
gmp.h	GMP header file, used by the C model
run_bitacc_cmodel.c	Example program for calling the C model
cic_compiler_v4_0_bitacc_mex.cpp	MATLAB MEX function source
make_cic_compiler_v4_0_mex.m	MATLAB MEX function compilation script
run_cic_compiler_v4_0_mex.m	MATLAB MEX function example script
@cic_compiler_v4_0_bitacc	MATLAB MEX function class directory

Installation

Linux

- Unpack the contents of the ZIP file.

- Ensure that the directory where the `libIp_cic_compiler_v4_0_bitacc_cmodel.so`, `libgmp.so.11`, and `libgmpxx.so.4` files reside is included in the path of the environment variable `LD_LIBRARY_PATH`.

Windows

- Unpack the contents of the ZIP file.
- Ensure that the directory where the `libIp_cic_compiler_v4_0_bitacc_cmodel.dll` and `libgmp.dll` files reside is:
 - included in the path of the environment variable `PATH` or
 - the directory in which the executable that calls the C model is run.

C Model Interface

The application programming interface (API) of the C model is defined in the header file `cic_compiler_v4_0_bitacc_cmodel.h`. The interface consists of data types and functions as described in the following sections.

An example file, `run_bitacc_cmodel.c`, is included; this demonstrates how to call the C model.

Data Types

The following table shows the types defined for the CIC Compiler C model.

Table 5: C Model Data Types

Name	Type	Description
<code>xip_real</code>	Double	Used for data sample values input to the CIC and for the double-precision data output.
<code>xip_cic_v4_0_config</code>	Struct	Structure of configuration parameters.
<code>xip_cic_v4_0_rate_packet</code>	Struct	Data structure of the rate value input to the core at runtime for programmable-rate filters.
<code>xip_status</code>	Int	Error code returned from many C model functions. 0 indicates success. Any other value indicates failure.
<code>xip_array_real</code>	Struct	Structure used to hold data for input to the CIC
<code>xip_array_mpz</code>	Struct	Structure used to hold the arbitrary-precision (mpz) integer data output from the CIC.
<code>xip_cic_v4_0</code>	Struct	Type defined which C (not C++) can use as a handle (pointer) to a C++ object – the C model itself.

`xip_cic_v4_0_config` is the configuration of the core itself. The members of this structure are listed in the `cic_compiler_v4_0_bitacc_cmodel` file. The names closely match the same names in the XCO/XCI files. The `cic_compiler_v4_0_bitacc_cmodel` file also contains `#define` values for enumerated fields.

`xip_array_real` and `xip_array_mps` are structures whose principal members are:

- **data:** A pointer to the array of data values.
- **data_size:** Of type `size_t`, which describes the number of valid elements in the data array.
- **dim:** A pointer to a `size_t` array of values that indicate the size of each dimension.
- **dim_size (size_t):** Indicates the number of dimensions of the data array.

Data Format

The C model holds input and output data in multi-dimensional arrays: `xip_array_real` for input and `xip_array_mps` for output.

Data is arranged as a 3D array, where `dim[0]` is the path (always 1), `dim[1]` is the number of channels, and `dim[2]` is the number of samples per channel.

Multiple channels are interleaved in the flat `data` buffer: all channels of a given sample are stored consecutively before the next sample. For three channels this is:

```
sample 0      sample 1      sample 2
   c0  c1  c2      c0  c1  c2      c0  c1  c2
```

The flat index of a given value is therefore `(sample * num_channels) + chan`.

To access a single value by channel and sample number without computing the flat index, use the helpers:

- `xip_cic_v4_0_xip_array_real_set_sample` writes an input sample.
- `xip_cic_v4_0_xip_array_mps_get_sample` reads an output sample.

Functions

The functions of the CIC Compiler C model are grouped into information, initialization, and execution functions.

Initialization Functions

The following table lists the functions to create, configure, and destroy the C model and associated data structures.

Name	Return	Arguments	Description
xip_cic_v4_0_default_config	xip_status	Pointer to configuration structure	Fills the configuration structure with the core's default values. Call this before overriding individual fields.
xip_cic_v4_0_create	Pointer to xip_cic_v4_0 (the C model itself)	Pointer to configuration structure, message handler, message handle	Creates a new C model object and returns a pointer to it. The message handler reports errors and warnings; pass a null pointer to suppress messages. Returns a null pointer on failure.
xip_cic_v4_0_reset	xip_status	Pointer to xip_cic_v4_0 (the C model itself)	Clears all internal filter state. For programmable-rate filters, the active rate is restored to the initial rate in the configuration.
xip_cic_v4_0_destroy	xip_status	Pointer to xip_cic_v4_0 (the C model itself)	Deallocates memory owned by the C model and destroys the C model itself.
xip_array_real_create / xip_array_mpz_create	Pointer to created data structure	None	Allocates memory for the data structure itself, not the array members within it. Use the real form for input arrays and the mpz form for output arrays.
xip_array_real_reserve_data / xip_array_mpz_reserve_data	xip_status	Pointer to data structure, maximum number of elements in data array	(Re)allocates enough memory for the maximum size.
xip_array_real_reserve_dim / xip_array_mpz_reserve_dim	xip_status	Pointer to data structure, maximum number of dimensions	Allocates the array that holds the size of each dimension of the data array. For example, 1 path x 4 channels x 100 samples.
xip_array_real_destroy / xip_array_mpz_destroy	xip_status	Pointer to data structure	Frees the memory allocated for the data array, the dimension array, and the data structure itself.

Information Functions

The following table lists the functions to query the C model.

Name	Return	Arguments	Description
xip_cic_v4_0_get_version	Pointer to version string	None	Returns the version of the C model as a string.
xip_cic_v4_0_get_config	xip_status	Pointer to C model, pointer to configuration structure	Copies the current configuration of the C model into the designated configuration structure, including the automatically computed per-stage bit widths.

Execution Functions

The following table lists the functions to send data to and retrieve results from the C model.

Name	Return	Arguments	Description
xip_cic_v4_0_calc_size_mpz	xip_status	Pointer to C model, pointer to input array, pointer to output array	Calculates the output size for a given input and populates the dim, dim_size, and data_size fields of the output array. Decimation output is the input count divided by the rate; interpolation output is the input count multiplied by the rate.
xip_cic_v4_0_data_send	xip_status	Pointer to C model, pointer to input array	Passes an input data packet to the model for processing. Any output from a previous call that has not been retrieved is discarded.
xip_cic_v4_0_data_get	xip_status	Pointer to C model, pointer to output array	Retrieves the output as double-precision values. Exact for output widths up to 52 bits; for wider outputs use xip_cic_v4_0_data_get_mpz.
xip_cic_v4_0_data_get_mpz	xip_status	Pointer to C model, pointer to output array	Retrieves the output as arbitrary-precision integers, exact at any output width.
xip_cic_v4_0_rate_send	xip_status	Pointer to C model, pointer to rate packet	For programmable-rate filters only. Applies a new rate, which takes effect on the next channel boundary. The rate must lie within the configured minimum and maximum.
xip_cic_v4_0_set_data_sink_mpz	xip_status	Pointer to C model, pointer to output array	Registers an output array into which the model writes its results, instead of retrieving them with a data_get call.
xip_cic_v4_0_set_data_handler	xip_status	Pointer to C model, data handler callback, handle, optional argument	Registers a callback that is invoked when the output sink is filled. Used to pipe the output into a downstream model.
xip_cic_v4_0_data_send_handler	int	Pointer to input array, pointer to C model (as void), unused argument	Callback form of data_send. Used as the data handler of an upstream model so its output is fed directly into this model.

To read output values directly, use xip_cic_v4_0_data_get_mpz.

Compiling

Compilation of user code requires access to the `cic_compiler_v4_0_bitacc_cmodel.h` header file. The header file needs to be copied to a location where it is available to the compiler. Depending on the location chosen, the include search path of the compiler might need to be modified.

When compiling on Windows, the symbol `NT` must be defined, either by a compiler option or in user source code before the `cic_compiler_v4_0_bitacc_cmodel.h` header file is included.

Linking

Linux

To use the C model, the user executable must be linked against the correct libraries for the target platform.

The executable must be linked against the following shared object libraries:

- `libgmp.so.11`
- `libIp_cic_compiler_v4_0_bitacc_cmodel.so`

Using GCC, linking is typically achieved by adding the following command line options:

```
-L. -Wl,-rpath,. -lIp_cic_compiler_v4_0_bitacc_cmodel
```

This assumes the shared object libraries are in the current directory. If this is not the case, the `-L.` option needs to be changed to specify the library search path to use.

Using GCC, the provided example program, `run_bitacc_cmodel.c`, can be compiled and linked using the following command:

```
gcc -x c++ -I. -L. -lIp_cic_compiler_v4_0_bitacc_cmodel -Wl,-rpath,. \  
-o run_bitacc_cmodel run_bitacc_cmodel.c
```

Note: The C model dynamically links to `libgmpxx.so.4`, and therefore, it must be visible to the model while running.

Windows

The executable must be linked against the following dynamic link libraries:

- `libgmp.dll`

- `libIp_cic_compiler_v4_0_bitacc_cmodel.dll`

Depending on the compiler, the import libraries might also be required:

- `libgmp.lib`
- `libIp_cic_compiler_v4_0_bitacc_cmodel.lib`

Using Microsoft Visual Studio, linking is typically achieved by adding the import libraries to the Additional Dependencies entry under the Linker section of Project Properties.

Example

The `run_bitacc_cmodel.c` file contains example code that shows the basic operation of the C model. The file demonstrates querying the model version, configuring and creating the model, allocating the required input and output data structures, sizing the output, executing the model, retrieving the results, deallocating the data structures, and destroying the model.

The results are printed to the standard output channel (stdout), and the fixed-rate cases are checked against known impulse-response values to report a pass or fail result.

MATLAB Interface

A MATLAB software interface to the C model is provided in the form of a MEX function and a MATLAB class. The class wraps the MEX function and presents the C model as a MATLAB object, allowing the model to be configured and run from MATLAB without calling the C API directly.

The interface is delivered in the same ZIP file as the C model and consists of:

- `cic_compiler_v4_0_bitacc_mex.cpp`: The MEX function source
- `make_cic_compiler_v4_0_mex.m`: The MEX build script
- `run_cic_compiler_v4_0_mex.m`: An example script
- `@cic_compiler_v4_0_bitacc`: The MATLAB class directory

Compiling

Source code for a MATLAB MEX function is provided. This can be compiled within MATLAB by changing to the directory which contains the code and running the `make_cic_compiler_v4_0_mex.m` script.

Installation

To use the MEX function the compiled MEX function must be present on the MATLAB search path. This can be achieved by either of the following:

- Adding the directory where the compiled MEX function is located to the MATLAB search path (refer to the MATLAB `addpath` function), or
- Copying the files to a location already on the MATLAB search path.

As with all uses of the C model, the correct C model libraries also need to be present on the platform library search path (that is, `PATH` or `LD_LIBRARY_PATH`).

MATLAB Class Interface

The `@cic_compiler_v4_0_bitacc` class handles the create/destroy semantics of the C model and presents it as a MATLAB object. Configuration field names and valid values are identical to those described for the C model in Data Types. All data and configuration values use the MATLAB type `double`. MATLAB arrays are used for the mapping of types as in the following table.

Table 6: C Model Type Mapping

C Model Type	MATLAB Type
<code>xip_real</code>	<code>double</code>
<code>xip_mpz</code>	<code>double</code>

The class provides the following methods.

Constructor

```
[model] = cic_compiler_v4_0_bitacc
[model] = cic_compiler_v4_0_bitacc(config)
[model] = cic_compiler_v4_0_bitacc(field, value [, field, value]*)
```

* indicates an optional parameter.

The first version of the function call constructs a model object using the default configuration. The second version constructs a model object from a structure that specifies the configuration parameter values to use. The third version is the same as the second, but allows the configuration to be specified as a series of (parameter name, value) pairs rather than a single structure. The names and valid values of configuration parameters are identical to those described for the C model in Data Types.

The MATLAB configuration structure can contain an additional element, `PersistentMemory`. When this element is set to `TRUE`, the internal filter state of the model is retained following a call to the filter function. Otherwise, the model is reset after the data is returned. `PersistentMemory` is set to `FALSE` by default.

Get Version

```
[version] = get_version(model)
```

This method returns the version string of the C model library used.

Get Configuration

```
[config] = get_configuration(model)
```

This method returns the current configuration structure of a model object, including the automatically computed per-stage widths.

Filter

```
[data_out] = filter(model, data_in)
```

This method passes input data through the CIC filter and returns the output. The input is a real, integer-valued double array: `[1 x num_samples]` for a single channel, or `[num_channels x num_samples]` for multiple channels. The output has the corresponding shape, where the number of output samples is the number of input samples divided by the rate for decimation, or multiplied by the rate for interpolation.

Rate Send

```
rate_send(model, rate)
```

This method applies a new rate to the model. It applies only when the model has been configured for programmable rate, and the new rate takes effect on the next channel boundary.

Reset

```
reset(model)
```

This method resets the internal filter state of the model.

Delete

```
delete(model)
```

This method destroys all memory associated with the model. It is called automatically when the model object goes out of scope, so it does not normally need to be called directly.

Example

The `run_cic_compiler_v4_0_mex.m` script contains example code that shows how to create and run the model from MATLAB. It constructs a model with the class constructor, passes data through it with the `filter` method, and prints a result for each test, displaying `Test completed successfully` when all tests pass.

The script covers fixed-rate decimation, fixed-rate interpolation, multi-channel decimation, and a programmable-rate case that uses `rate_send` with `PersistentMemory` enabled.

Test Bench

This chapter contains information about the provided test bench in the AMD Vivado™ Design Suite.

Demonstration Test Bench

When the core is generated using IP catalog, a demonstration test bench is created. This is a simple VHDL test bench that exercises the core.

The demonstration test bench source code is one VHDL file: `demo_tb/tb_<component_name>.vhd` in the IP catalog output directory. The source code is comprehensively commented.

Using the Demonstration Test Bench

The demonstration test bench instantiates the generated CIC Compiler core. Compile the netlist and the demonstration test bench into the work library (see your simulator documentation for more information on how to do this). Then simulate the demonstration test bench. View the test bench signals in your simulator waveform viewer to see the operations of the test bench.

Demonstration Test Bench in Detail

The demonstration test bench performs these tasks

- Instantiate the core
- Generate a clock signal
- Drive the core clock enable and reset input signals (if present)
- Drive the core input signals to demonstrate core features (see the following sections for details)
- Provide signals showing the separate fields of AXI4 `TDATA` and `TUSER` signals

The demonstration test bench drives the core input signals to demonstrate the features and modes of operation of the core. The test bench drives a sine wave into the CIC core. In multichannel mode, the frequency of the sine wave increases with each channel. The output of the core shows the same sine wave (or waves in multichannel mode) but in a decimated or interpolated format. Alias signals are used to decode the AXI4 channels and allow easy viewing of the input and output data.

The operations performed by the demonstration test bench are appropriate for the configuration of the generated core, and are a subset of the following operations (in order):

1. Asserts reset at the start of the test. Only available when the core is configured with a reset (`HAS_ARESETN = TRUE`).
2. Sends data to CIC Compiler core with no waitstates. The upstream master supplies data to the CIC Compiler core at the input sample rate.
3. Sends data to CIC Compiler core with waitstates. The upstream master adds random waitstates to the input samples by deasserting `s_axis_data_tvalid`.
4. Asserts and deasserts clock enable (`aclken`) on alternating clock cycles. This has the effect of halving the input sample rate. Only available when the core is configured with a clock enable (`HAS_ACLKEN = TRUE`).
5. Changes the rate of the core. Only available in programmable rate cores.
6. Injects waitstates on Data Output Channel. The downstream slave keeps `m_axis_data_tready` deasserted for random periods of time. Only available when the core is configured with a TREADY on the Data Output Channel (`HAS_DOUT_TREADY = TRUE`).

Customizing the Demonstration Test Bench

It is possible to modify the demonstration test bench to drive the core inputs with different data or to perform different operations. Input data generated on the fly using a function called `calculate_next_input_sample()`. By default, it generates a sine wave, but could be modified to generate an impulse (for example).

The clock frequency of the core can be modified by changing the `CLOCK_PERIOD` constant.

Upgrading

This appendix contains information about migrating a design from ISE to the AMD Vivado™ Design Suite, and for upgrading to a more recent version of the IP core. For customers upgrading in the AMD Vivado™ Design Suite, important details (where applicable) about any port changes and other impact to user logic are included.

Migrating to the Vivado Design Suite

Migrating to the Vivado Design Suite For information on migrating to the Vivado Design Suite, refer to the *ISE to Vivado Design Suite Migration Guide* ([UG911](#)).

Upgrading in the Vivado Design Suite

This section provides information about any changes to the user logic or port designations that take place when you upgrade to a more current version of this IP core in the Vivado Design Suite.

Parameter Changes in the XCO/XCI File

The IP catalog core update feature can be used to update an existing CIC Compiler XCO file to version v4.0 of the CIC Compiler core. The core can then be regenerated to create a new netlist. Refer to the Vivado IP catalog documentation for more information on this feature. The update mechanism alone does not create a core compatible with v2.0. Refer to [Instructions for Minimum Change Migration](#). CIC Compiler v4.0 has additional parameters for AXI4-Stream support. The following table shows the changes to XCO parameters from v3.0 and v4.0 have identical XCI parameters, ports, latency and behavior.

Table 7: XCO/XCI Parameter Changes from v2.0 to v4.0

Version 2.0	Version 4.0	Notes
GUI_Behaviour	GUI_Behaviour	Unchanged
Filter_Type	Filter_Type	Unchanged
Number_of_Stages	Number_of_Stages	Unchanged

Table 7: XCO/XCI Parameter Changes from v2.0 to v4.0 (cont'd)

Version 2.0	Version 4.0	Notes
Differential_Delay	Differential_Delay	Unchanged
Number_Of_Channels	Number_Of_Channels	Unchanged
Sample_Rate_Changes	Sample_Rate_Changes	Unchanged
Fixed_Or_Initial_Rate	Fixed_Or_Initial_Rate	Unchanged
Minimum_Rate	Minimum_Rate	Unchanged
Maximum_Rate	Maximum_Rate	Unchanged
RateSpecification	RateSpecification	Unchanged
Input_Sample_Frequency	Input_Sample_Frequency	Unchanged
Clock_Frequency	Clock_Frequency	Unchanged
HardwareOversamplingRate	HardwareOversamplingRate	Unchanged
SamplePeriod	SamplePeriod	Unchanged
Passband_Min	Passband_Min	Unchanged
Stopband_Min	Stopband_Min	Unchanged
Passband_Max	Passband_Max	Unchanged
Stopband_Max	Stopband_Max	Unchanged
Input_Data_Width	Input_Data_Width	Unchanged
Output_Data_Width	Output_Data_Width	Unchanged
Quantization	Quantization	Unchanged
CE	HAS_ACLKEN	Renamed
SCLR	HAS_ARESETN	Renamed
ND	ND	Unchanged
Use_Streaming_Interface	Use_Streaming_Interface	Unchanged
Use_Xtreme_DSP_Slice	Use_Xtreme_DSP_Slice	Unchanged
—	HAS_DOUT_TREADY	New

Port Changes

The following table details the changes to port naming, additional or deprecated ports and polarity changes from v2.0 to v4.0:

Table 8: Port Changes from Version 2.0 to Version 4.0

Version 2.0	Version 4.0	Notes
CLK	ACLK	Renamed
CE	ALCKEN	Renamed
SCLR	ARESETN	Renamed Polarity change (now active-Low) Minimum length now two clock cycles
DIN	S_AXIS_DATA_TDATA	Renamed
ND	S_AXIS_DATA_TVALID	Renamed

Table 8: Port Changes from Version 2.0 to Version 4.0 (cont'd)

Version 2.0	Version 4.0	Notes
RFD	S_AXIS_DATA_TREADY	Renamed
—	S_AXIS_DATA_TLAST	New signal
RATE	S_AXIS_CONFIG_TDATA	Renamed
RATE_WE	S_AXIS_CONFIG_TVALID	Renamed
—	S_AXIS_CONFIG_TREADY	New signal
DOUT	M_AXIS_DATA_TDATA	Renamed
RDY	M_AXIS_DATA_TVALID	Renamed
—	M_AXIS_DATA_TREADY	New signal Optional
—	M_AXIS_DATA_TLAST	New signal
CHAN_SYNC	M_AXIS_DATA_TUSER	Renamed
CHAN_OUT	M_AXIS_DATA_TUSER	Renamed
—	event_tlast_missing	New signal
—	event_tlast_unexpected	New signal
—	event_halted	New signal

Functionality Changes

Latency Changes

- **HAS_DOUT_TREADY = 0:** The latency is identical to the equivalent configuration of v2.0.
- **HAS_DOUT_TREADY = 1:** The latency is variable, with only the minimum possible latency being determinable. The latency is a minimum of three cycles longer than the equivalent configuration of v2.0. The update process cannot account for this and guarantee equivalent performance.

Simulation

Starting with CIC Compiler v4.0 (2013.3 version), behavioral simulation models are replaced with IEEE P1735 Encrypted VHDL. The resulting model is bit and cycle accurate with the final netlist. For more information on simulation, see the *Vivado Design Suite User Guide: Logic Simulation* (UG900).

Instructions for Minimum Change Migration

To configure the CIC Compiler v4.0 to most closely mimic the behavior of v2.0, apply the following translation:

- **XCO Parameters:** Set HAS_DOUT_TREADY to 0. This makes the interface equivalent to v2.0 with ND enabled.

- `s_axis_data_tvalid` is equivalent to ND
- `s_axis_data_tready` is equivalent to RFD
- `m_axis_data_tvalid` is equivalent to RDY

If ND was previously disabled (ND XCO parameter was FALSE), then tie `s_axis_data_tvalid` to 1. This means data can always be supplied, and the data transfer is solely controlled by `s_axis_data_tready` (previously RFD).

If SCLR was previously set to TRUE, remember that the reset pulse is now active-Low and must be a minimum of two clock cycles long.

Special Considerations when Migrating to AXI

The conversion to AXI4-Stream interfaces brings standardization and enhances interoperability of AMD IP LogiCORE solutions. Other than general control signals such as `aclk`, `aclken` and `aresetn`, and event signals, all inputs and outputs to the CIC Compiler core are conveyed using AXI4-Stream channels. A channel always consists of TVALID and TDATA plus additional ports (such as TREADY, TUSER and TLAST) when required and optional fields. Together, TVALID and TREADY perform a handshake to transfer a message, where the payload is TDATA, TUSER and TLAST.

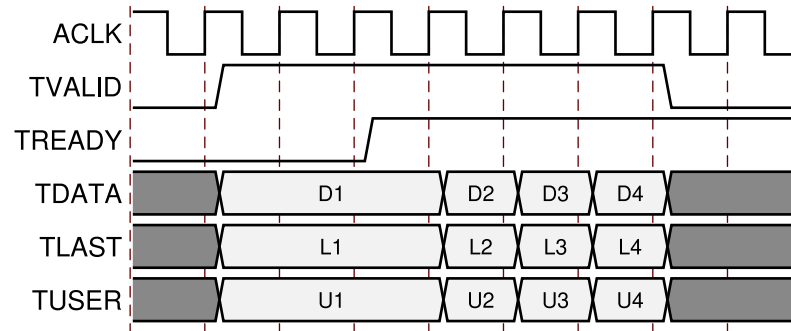
For further details on AXI4-Stream interfaces, refer to the *Vivado Design Suite: AXI Reference Guide* ([UG1037](#)).

Basic Handshake

The following figure shows the transfer of data in an AXI4-Stream channel:

- TVALID is driven by the source (master) side of the channel and TREADY is driven by the receiver (slave).
- TVALID indicates that the values in the payload fields (TDATA, TUSER and TLAST) are valid.
- TREADY indicates that the slave is ready to receive data.
- When both TVALID and TREADY are TRUE in a cycle, a transfer occurs.
- The master and slave set TVALID and TREADY respectively for the next transfer appropriately.

Figure 26: Data Transfer in an AXI4-Stream Channel



AXI4 Channel Rules

All of the AXI4 channels follow the same fundamental rules:

- All TDATA and TUSER fields are packed in little endian format. Bit 0 of a sub-field is aligned to the same side as bit 0 of TDATA or TUSER.
- All TDATA and TUSER vectors are multiples of 8 bits. After all fields in a TDATA or TUSER vector are concatenated, the overall vector is padded to bring it up to an 8 bit boundary.

Configuration Channel

This channel is only present in programmable rate configurations.

Pinout

The configuration channel pinout is shown in the following table.

Table 9: Configuration Channel Pinout

Port Name	Port Width	Direction	Description
s_axis_config_tdata	Variable ¹	In	Carries the configuration information: RATE
s_axis_config_tvalid	1	In	Asserted by the external master to signal that it is able to provide data.
s_axis_config_tready	1	Out	Asserted by the CIC Compiler to signal that it is able to accept data.

Notes:

1. Range can be viewed in the Vivado IDE.

TDATA Fields

The configuration channel (`s_axis_config`) is an AXI4 channel that carries the fields shown in the following table in its TDATA vector.

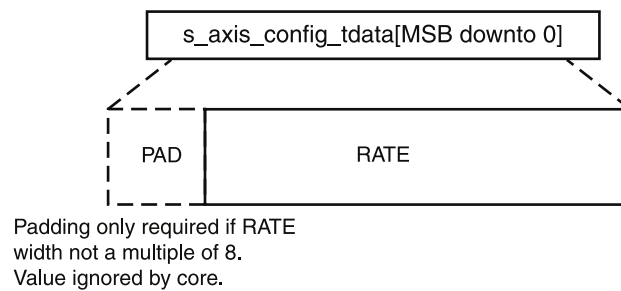
Table 10: Configuration Channel TDATA Fields

Field Name	Width	Padded	Description
RATE	$\log_2(\text{maximum rate} + 1)$	Yes	The interpolation or decimation rate of the core.

The RATE field needs to be extended to the next 8-bit boundary if it does not already finish on an 8-bit boundary. The CIC Compiler core ignores the value of the padding bits, so they can be driven to any value. Connecting them to constant values can help reduce device resource usage.

TDATA Format

The configuration channel has one field which is packed into the `s_axis_config_tdata` vector as shown in the following figure (starting from the LSB):

Figure 27: Configuration Channel TDATA (`s_axis_config_tdata`) Format


PG140_c3_02_011413

Data Input Channel

The Data Input Channel contains the sample data to be transformed.

Pinout

The data input channel pinout is shown in the following table.

Table 11: Data Input Channel Pinout

Port Name	Port Width	Direction	Description
<code>s_axis_data_tdata</code>	Variable ¹	In	Carries the sample data
<code>s_axis_data_tvalid</code>	1	In	Asserted by the upstream master to signal that it is able to provide data
<code>s_axis_data_tlast</code>	1	In	Asserted by the upstream master on the sample corresponding to the last channel. This is not used by the CIC Compiler except to generate the events: <code>event_tlast_unexpected</code> <code>event_tlast_missing</code>. Only available when the core is configured to have multiple channels.

Table 11: Data Input Channel Pinout (cont'd)

Port Name	Port Width	Direction	Description
s_axis_data_tready	1	Out	Used by the CIC Compiler to signal that it is ready to accept data

Notes:

1. Range can be viewed in the Vivado IDE.

TDATA Fields

The DATA field is packed into the `s_axis_data_tdata` vector as shown in the following table.

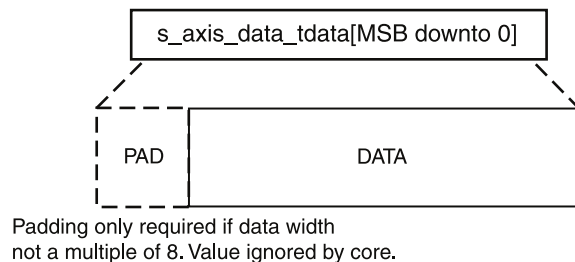
Table 12: Data Input Channel TDATA Fields

Field Name	Width	Padded	Description
DATA	2 to 32	Yes	The sample data to be processed in twos complement format.

The DATA field needs to be extended to the next 8-bit boundary if it does not already finish on an 8-bit boundary. The CIC Compiler core ignores the value of the padding bits, so they can be driven to any value. Connecting them to constant values can help reduce device resource usage.

TDATA Format

The data input channel TDATA vector (`s_axis_data_tdata`) has one field (DATA) which is packed as shown in Figure A-3.

Figure 28: Data Input Channel TDATA (`s_axis_data_tdata`) Format

PG140_c3_03_011413

Data Output Channel

The data output channel contains the processed samples, which are carried on TDATA. In addition, TUSER carries per-sample status information relating to the sample data on TDATA. This status information is intended for use by downstream slaves that directly process data samples. It cannot get out of synchronization with the data as it is transferred in the same channel. This information is classed as per-sample status:

1. CHAN_SYNC

2. CHAN_OUT

Pinout

The data output channel pinout is shown in the following table.

Table 13: Data Output Channel Pinout

Port Name	Port Width	Direction	Description
m_axis_data_tdata	Variable ¹	Out	Carries the sample data.
m_axis_data_tuser	Variable ¹	Out	Carries additional per-sample status information. Only available when the core is configured to have multiple channels.
m_axis_data_tvalid	1	Out	Asserted by the CIC Compiler to signal that it is able to provide sample data.
m_axis_data_tlast	1	Out	Asserted by the CIC Compiler on the sample corresponding to the last channel. Only available when the core is configured to have multiple channels.
m_axis_data_tready	1	In	Asserted by the external slave to signal that it is ready to accept data. Only available when the core is configured to have a TREADY on the data output channel.

Notes:

1. Range can be viewed in the Vivado IDE.

TDATA Fields

The DATA field is packed into the `m_axis_data_tdata` vector as shown in the following table.

Table 14: Data Output Channel TDATA Fields

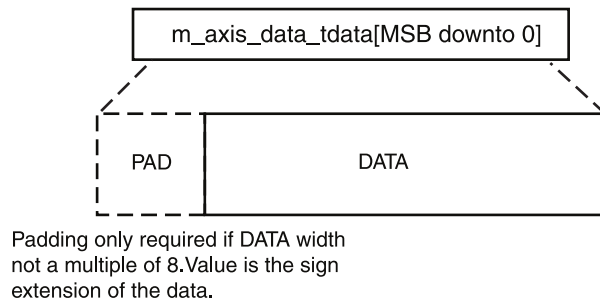
Field Name	Width	Padded	Description
DATA	2 to 104	Yes – sign extended	The processed sample data in twos complement format.

The DATA field is sign extended to the next 8-bit boundary if it does not already finish on an 8-bit boundary.

TDATA Format

The data output channel TDATA vector (`m_axis_data_tdata`) has one field (DATA) which is packed as shown in the following figure.

Figure 29: Data Output Channel TDATA (m_axis_data_tdata) Format



PG140_c3_04_011413

TUSER Fields

The data output channel carries the fields shown in the following table in its TUSER vector.

Table 15: Data Output Channel TUSER Fields

Field Name	Width	Padded	Description
CHAN_OUT	log2(number of channels)	Yes – zero-extended	The number of the channel (0 indexed) corresponding to the sample on m_axis_data_tdata.
CHAN_SYNC	1	Yes – zero-extended	Asserted with the sample corresponding to the first channel. Only available when the core is configured to have multiple channels.

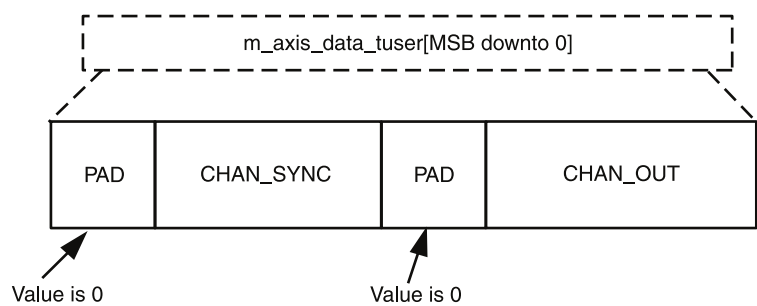
All fields with padding need to be zero-extended to the next 8-bit boundary if they do not already finish on an 8-bit boundary.

TUSER Format

The fields are packed into the `m_axis_data_tuser` vector in the order shown in the following figure (starting from the LSB):

1. CHAN_OUT plus padding
2. CHAN_SYNC plus padding

Figure 30: **Data Output Channel TUSER (m_axis_data_tuser) Format**



PG140_c3_05_011413

Debugging

This appendix includes details about resources available on the AMD Support website and debugging tools.

If the IP requires a license key, the key must be verified. The AMD Vivado™ design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)



IMPORTANT! IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.

Finding Help with AMD Adaptive Computing Solutions

To help in the design and debug process when using the core, the [Support web page](#) contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The [Community Forums](#) are also available where members can learn, participate, share, and ask questions about AMD Adaptive Computing solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the [AMD Adaptive Support web page](#) or by using the AMD Adaptive Computing Documentation Navigator. Download the Documentation Navigator from the [Downloads page](#). For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with an AMD Adaptive Computing product. Answer Records are created and maintained daily to ensure that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main [AMD Adaptive Support web page](#). To maximize your search results, use keywords such as:

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Master Answer Record for the Core

AR [29297](#).

Technical Support

AMD Adaptive Computing provides technical support on the [Community Forums](#) for this AMD LogiCORE™ IP product when used as described in the product documentation. AMD Adaptive Computing cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the [Community Forums](#).

Debug Tools

There are many tools available to address CIC Compiler design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The AMD Vivado™ Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in AMD devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#)).

Interface Debug

If data is not being transmitted or received, check the following conditions:

- If transmit `interface_name_tready` is stuck Low following the `interface_name_tvalid` input being asserted, the core cannot send data.
- If the receive `interface_name_tvalid` is stuck Low, the core is not receiving data.
- Check that the `ACLK` inputs are connected and toggling.
- Check that the AXI4-Stream waveforms are being followed. Refer to [Protocol Description](#).
- Check the core configuration.

Additional Resources and Legal Notices

Finding Additional Documentation

Technical Information Portal

The AMD Technical Information Portal is an online tool that provides robust search and navigation for documentation using your web browser. To access the Technical Information Portal, go to <https://docs.amd.com>.

Documentation Navigator

Documentation Navigator (DocNav) is an installed tool that provides access to AMD Adaptive Computing documents, videos, and support resources, which you can filter and search to find information. To open DocNav, do the following:

- From the AMD Vivado™ IDE, select **Help** → **Documentation and Tutorials**.
- On Windows, click the **Start** button and select **AMD Adaptive SoC and FPGA Tools** → **DocNav**.
- At the Linux command prompt, enter `docnav`.

Note: For more information on DocNav, refer to the *Documentation Navigator User Guide* ([UG968](#)).

Design Hubs

AMD Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs, do the following:

- In DocNav, click the **Design Hubs View** tab.
- Go to the [Design Hubs](#) web page.

Support Resources

For support resources such as Answers, Documentation, Downloads, and Forums, refer to [Support](#).

References

These documents provide supplemental material useful with this guide:

1. Eugene B. Hogenauer, *An Economical Class of Digital Filters for Decimation and Interpolation*, IEEE Transactions on Acoustics, Speech, and Signal Processing, Vol.ASSP-29, No. 2, April 1981.
 2. *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
 3. *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
 4. *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))
 5. *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
 6. *ISE to Vivado Design Suite Migration Guide* ([UG911](#))
 7. *Vivado Design Suite User Guide: Release Notes, Installation, and Licensing* ([UG973](#))
 8. *Vivado Design Suite: AXI Reference Guide* ([UG1037](#))
 9. *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#))
-

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
07/22/2026 Version 4.0	
Chapter 6: C Model	Added chapter.
02/04/2021 Version 4.0	
N/A	Added Versal adaptive SoC support.
05/10/2016 Version 4.0	
N/A	Increased range of input data.
11/18/2015 Version 4.0	
N/A	Added support for UltraScale+ families.

Section	Revision Summary
09/30/2015 Version 4.0	
N/A	- Corrected signal direction in Product Specification chapter. - Corrected range information in Design Flow Steps chapter.
04/02/2014 Version 4.0	
N/A	Added link to resource utilization information.
12/08/2013 Version 4.0	
N/A	- Added UltraScale architecture support information. - Added Simulation, Synthesis, and Test Bench chapters. - Updated Migrating appendix.
10/02/2013 Version 4.0	
N/A	Minor updates to IP Facts table and Migrating appendix. Document version number advanced to match the core version number.
03/20/2013 Version 1.0	
Initial release.	This document derived from the LogiCORE CIC Compiler Data Sheet (DS845).

Please Read: Important Legal Notices

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes. THIS INFORMATION IS PROVIDED "AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2013-2026 Advanced Micro Devices, Inc. AMD, the AMD Arrow logo, UltraScale, UltraScale+, Versal, Vivado, Zynq, and combinations thereof are trademarks of Advanced Micro Devices, Inc. MATLAB and Simulink are registered trademarks of The MathWorks, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.